

IF2224 TEORI BAHASA FORMAL DAN OTOMATA
LAPORAN TUGAS BESAR
MILESTONE 4



Disusun oleh:

Kelompok 01 – Kelas 01

Muhammad Jordan Ferimeison (13524047)

Arina Azka (13524049)

Hakam Avicena Mustain (13524075)

Yavie Azka Putra Araly (13524077)

Angelina Andra Alanna (13524079)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

BAB I

LANDASAN TEORI

1.1. Arsitektur Front-end

Dalam perancangan kompiler dan interpreter modern, arsitektur sistem secara umum dibagi menjadi dua fase utama, yaitu front-end dan back-end. Front-end merupakan fase yang berinteraksi langsung dengan kode sumber yang ditulis oleh pemrogram. Tanggung jawab utama front-end mencakup tiga tahap yang dijalankan secara berurutan.

Tahap pertama adalah analisis leksikal. Pada tahap ini, teks mentah kode sumber dipecah menjadi unit-unit bermakna terkecil yang disebut token. Setiap token merepresentasikan satu elemen bahasa, seperti kata kunci, identifier, literal, atau operator. Proses ini dijalankan oleh komponen yang disebut lexer atau scanner, yang pada umumnya diimplementasikan menggunakan Deterministic Finite Automaton (DFA).

Tahap kedua adalah analisis sintaksis. Pada tahap ini, deretan token yang dihasilkan oleh lexer disusun ke dalam struktur hierarkis berupa parse tree atau Abstract Syntax Tree (AST). Proses ini memvalidasi bahwa urutan token sesuai dengan aturan grammar bahasa pemrograman yang bersangkutan.

Tahap ketiga adalah analisis semantik. Pada tahap ini, AST divalidasi dari sisi kebenaran logika dan tipe data. Proses ini mencakup pengecekan tipe, resolusi identifier melalui tabel simbol, serta validasi kompatibilitas operasi. Hasil akhir dari tahap ini adalah Decorated AST, yaitu AST yang telah dianotasi dengan informasi semantik seperti tipe data setiap ekspresi, referensi ke entri tabel simbol, dan lexical level.

Secara keseluruhan, front-end bertanggung jawab mengubah teks mentah menjadi representasi terstruktur dan tervalidasi yang siap diproses oleh back-end. Pemisahan ini penting karena memungkinkan front-end untuk diganti tanpa mengubah back-end selama keluaran yang dihasilkan tetap konsisten.

1.2. Arsitektur Back-end

Back-end merupakan fase yang menerima keluaran dari front-end berupa Decorated AST dan bertanggung jawab menerjemahkannya menjadi instruksi yang dapat dieksekusi. Berbeda dengan front-end yang memperhatikan sintaksis bahasa, back-end sama sekali

tidak memperhatikan representasi tekstual program. Fokus back-end terletak murni pada dua hal, yaitu sintesis dan eksekusi.

Sintesis adalah proses mengonversi Decorated AST menjadi kode perantara (intermediate code), yaitu daftar instruksi operasional standar yang bersifat sekuensial dan tidak bergantung pada platform tertentu. Proses ini dilakukan oleh komponen yang disebut Intermediate Code Generator (ICG), yang pada umumnya menggunakan pendekatan penelusuran mendalam (Depth-First Search) pada AST.

Eksekusi adalah proses menjalankan kode perantara tersebut pada sebuah mesin virtual. Komponen yang bertanggung jawab atas proses ini dapat berupa interpreter yang mengeksekusi kode perantara secara langsung, atau code generator yang menerjemahkannya lebih lanjut menjadi bahasa mesin untuk platform tertentu.

Dengan demikian, jika suatu bahasa pemrograman ingin diubah sintaksnya, hanya front-end yang perlu dirombak. Selama front-end baru tersebut menghasilkan Decorated AST dengan format yang sama, seluruh mesin back-end dapat digunakan kembali tanpa modifikasi.

1.3. Three-Address Code (arin)

Three-Address Code (TAC) adalah format representasi kode perantara yang paling luas digunakan dalam perancangan kompiler. Format ini dinamakan demikian karena setiap instruksinya didesain sangat sederhana, dengan maksimal hanya melibatkan tiga alamat memori, yaitu dua operan sumber dan satu lokasi tujuan. Struktur dasar TAC mengikuti pola hasil = operan1 operator operan2.

Tujuan utama TAC adalah menjembatani kesenjangan antara representasi tingkat tinggi berupa AST dan instruksi tingkat rendah yang dapat dieksekusi oleh mesin. TAC meratakan struktur pohon yang kompleks menjadi daftar instruksi linier yang sekuensial sehingga lebih mudah dieksekusi oleh mesin virtual yang bekerja secara baris per baris.

Salah satu konsep penting dalam TAC adalah penggunaan variabel sementara. Ekspresi kompleks seperti $x := (a + b) * (c - d)$ tidak dapat dihitung dalam satu langkah oleh CPU sehingga TAC memecahnya menjadi operasi-operasi biner sederhana. Sebagai contoh, ekspresi tersebut dipecah menjadi empat instruksi:

- $t1 := a + b$ untuk mengevaluasi kurung pertama,

- $t2 := c - d$ untuk mengevaluasi kurung kedua,
- $t3 := t1 * t2$ untuk mengalikan kedua hasil, dan $x := t3$ untuk menyimpan hasil akhir.

Proses ini dilakukan melalui penelusuran post-order pada AST, yaitu cabang terdalam dievaluasi terlebih dahulu, kemudian hasilnya disimpan di variabel sementara yang digunakan oleh operasi di level atasnya.

Dalam implementasi berbasis stack machine, variabel sementara tidak perlu diberi nama secara eksplisit karena puncak stack berfungsi sebagai tempat penyimpanan hasil sementara secara implisit. Format instruksi pada stack machine umumnya terdiri atas empat kolom: nomor baris, opcode, level, dan operan. Jenis instruksi yang umum digunakan mencakup pemuatan literal, pemuatan dan penyimpanan nilai dari/ke alamat memori, operasi aritmatika dan perbandingan, lompatan bersyarat dan tanpa syarat, pemanggilan dan pengembalian dari subprogram, serta alokasi memori.

1.4. Translasi control flow ke TAC

Translasi control flow merupakan proses mengubah struktur kendali program seperti percabangan dan perulangan menjadi instruksi TAC yang bersifat linier dan sekuensial. Karena TAC tidak memiliki konsep blok bertingkat, seluruh kendali aliran program direpresentasikan melalui kombinasi instruksi lompatan bersyarat (JPC) dan lompatan tanpa syarat (JMP).

Instruksi JPC pada implementasi ini bersifat "lompat jika kondisi bernilai false (0)". Dengan kata lain, jika nilai di puncak stack adalah 0, eksekusi dialihkan ke alamat yang ditentukan. Jika nilainya 1 (true), eksekusi dilanjutkan ke instruksi berikutnya secara normal.

Translasi pernyataan IF-ELSE mengikuti pola berikut. Pertama, kondisi dievaluasi dan hasilnya ditempatkan di puncak stack. Kemudian disisipkan instruksi JPC dengan alamat tujuan yang masih kosong (placeholder). Selanjutnya instruksi untuk blok then di-generate. Jika terdapat blok else, disisipkan instruksi JMP untuk melewati blok else setelah blok then selesai dieksekusi. Setelah itu instruksi untuk blok else di-generate. Terakhir, alamat tujuan JPC dan JMP yang sebelumnya kosong di-patch dengan alamat

yang telah diketahui. Jika tidak terdapat blok else, JPC langsung di-patch ke instruksi setelah blok then.

Translasi pernyataan WHILE mengikuti pola berikut. Posisi instruksi awal loop dicatat sebelum kondisi di-generate. Kondisi dievaluasi dan hasilnya ditempatkan di puncak stack. Kemudian disisipkan JPC dengan alamat placeholder untuk keluar dari loop. Instruksi body loop di-generate. Setelah body selesai, disisipkan JMP yang mengarah kembali ke posisi awal loop. Terakhir, JPC di-patch ke instruksi pertama setelah loop.

Translasi pernyataan FOR pada dasarnya merupakan desugaring dari struktur WHILE. Nilai awal dievaluasi dan disimpan ke variabel loop. Posisi awal loop dicatat. Kondisi dievaluasi dengan membandingkan nilai variabel loop terhadap nilai batas menggunakan LEQ untuk TO atau GEQ untuk DOWNTO. JPC disisipkan sebagai pintu keluar. Body loop di-generate. Setelah body, variabel loop diinkremen atau didekremen sebesar satu, kemudian JMP kembali ke awal loop. JPC di-patch ke instruksi setelah loop. Translasi pernyataan REPEAT-UNTIL berbeda dari WHILE karena body selalu dieksekusi minimal satu kali sebelum kondisi dicek. Posisi awal loop dicatat sebelum body di-generate. Seluruh statement di dalam blok repeat di-generate. Kondisi until kemudian dievaluasi. Karena semantik REPEAT adalah "ulangi selama kondisi belum terpenuhi", JPC digunakan untuk kembali ke awal loop jika kondisi masih bernilai false. Jika kondisi bernilai true, eksekusi berlanjut ke instruksi setelah loop.

Translasi pernyataan CASE-OF menggunakan rangkaian perbandingan. Untuk setiap cabang, nilai selector dievaluasi ulang dan dibandingkan dengan nilai label menggunakan OPR EQL. Jika tidak cocok, JPC melompat ke perbandingan berikutnya. Jika cocok, JMP mengarahkan eksekusi ke body cabang yang bersangkutan. Di akhir setiap body disisipkan JMP untuk melompat ke instruksi setelah seluruh blok case. Semua alamat JMP dan JPC tersebut di-patch setelah seluruh cabang selesai di-generate.

1.5. Penggunaan Label & Jump

Dalam arsitektur stack machine yang digunakan pada proyek ini, tidak terdapat label bernama seperti yang dijumpai pada assembly konvensional. Sebagai gantinya, label direpresentasikan secara implisit sebagai nomor baris instruksi dalam daftar TAC.

Mekanisme jump bekerja dengan cara menetapkan nilai nomor baris tujuan sebagai operan instruksi JMP atau JPC.

Karena alamat tujuan lompatan seringkali belum diketahui saat instruksi jump di-emit, digunakan teknik yang disebut backpatching. Teknik ini bekerja dalam dua tahap. Pada tahap pertama, instruksi JMP atau JPC di-emit dengan operan bernilai 0 sebagai placeholder. Posisi instruksi tersebut dalam daftar disimpan ke dalam variabel sementara. Pada tahap kedua, setelah bagian kode yang menjadi tujuan lompatan selesai di-generate dan nomor baris tujuannya sudah diketahui, operan instruksi jump yang sebelumnya masih placeholder diperbarui langsung melalui akses ke elemen array instruksi.

Sebagai contoh, pada translasi IF-ELSE, JPC di-emit sebelum blok then di-generate. Posisinya disimpan di variabel jpcPos. Setelah blok then selesai, instruksi JMP di-emit untuk melewati blok else. Posisinya disimpan di variabel jmpPos. Blok else kemudian di-generate. Setelah blok else selesai, dua operasi patch dilakukan: `instructions[jpcPos].operand` diisi dengan nomor baris awal blok else, dan `instructions[jmpPos].operand` diisi dengan nomor baris instruksi setelah blok else.

Mekanisme backpatching ini memungkinkan ICG bekerja dalam satu kali penelusuran (single-pass) pada AST tanpa memerlukan tahap resolusi label terpisah. Seluruh koordinasi antara titik lompatan dan tujuannya diselesaikan secara lokal di dalam masing-masing fungsi gen, sehingga tidak ada ketergantungan global antar komponen ICG.

1.6. Decorated AST

Decorated AST adalah AST yang telah ditambahkan anotasi (dekorasi) pada node-node pohon untuk menyimpan informasi semantik yang diperoleh selama semantic analysis, seperti tipe ekspresi, alamat atau referensi ke entri symbol table, dan informasi lexical level atau offset. Anotasi ini membuat AST menjadi sumber tunggal kebenaran untuk informasi yang dibutuhkan tahap selanjutnya.

Decorated AST adalah AST yang telah ditambahkan anotasi (dekorasi) pada node-node pohon untuk menyimpan informasi semantik yang diperoleh selama semantic analysis, seperti tipe ekspresi, alamat atau referensi ke entri symbol table, dan informasi lexical level atau offset. Anotasi ini membuat AST menjadi sumber tunggal kebenaran

untuk informasi yang dibutuhkan tahap selanjutnya (mis. kode antara atau pengecekan tipe lanjut.

Decorated AST memungkinkan fase pembuatan kode antara dan back-end untuk membaca informasi semantik secara langsung dari AST tanpa mengulang analisis. Ini mempercepat generasi kode dan mengurangi kemungkinan inkonsistensi. Selain itu, annotated AST mempermudah optimasi berbasis tipe dan analisis liveness/aliasing.

Proses dekorasi dilakukan melalui transversal DFS pada AST menggunakan fungsi visit untuk setiap jenis node. Setiap fungsi visit bertanggung jawab untuk mendapatkan informasi semantik node tersebut, seperti mencari tipe data, melakukan lookup ke symbol table, dan memverifikasi kompatibilitas tipe, kemudian menyimpan hasilnya sebagai anotasi pada node yang bersangkutan.

<i>Tabel 1.6 Perbedaan AST dan Decorated AST</i>	
Sebelum Dekorasi (AST):	
	<pre> AssignNode -- VarNode("a") -- NumberNode(5) </pre>
Sesudah Dekorasi (Decorated AST):	
	<pre> AssignNode [type: void] -- VarNode("a") [type: Integer, tab_index: 34, lev: 0] -- NumberNode(5) [type: Integer] </pre>

1.7. Scope dan Symbol Table

Symbol table adalah struktur data sentral yang digunakan kompiler untuk menyimpan dan mengelola informasi tentang setiap identifier yang dideklarasikan dalam program, mencakup nama, jenis objek, tipe data, lexical level, dan alamat relatif dalam frame memori [3]. Symbol table diisi secara bertahap selama analisis semantik dan dibaca kembali selama pembangkitan kode.

1.7.1.Struktur Symbol Table

Pada kompiler berbasis blok seperti Pascal, symbol table umumnya terdiri dari beberapa tabel paralel:

- Identifier table (tab): menyimpan semua identifier yang dideklarasikan, termasuk kata kunci, variabel, konstanta, tipe, prosedur, dan fungsi. Setiap entri memuat nama, tipe, level, alamat, serta tautan ke entri sebelumnya dalam blok yang sama membentuk linked list per blok.
- Block table (btab): satu entri per blok atau scope; menyimpan informasi struktural sebuah blok seperti jumlah parameter (psze) dan jumlah variabel lokal (vsze), serta penunjuk ke entri terakhir di identifier table untuk blok tersebut.
- Array table (atab): menyimpan informasi tipe array secara terpisah, mencakup batas bawah, batas atas, ukuran elemen, dan ukuran total.

1.7.2.Lexical Scoping

Scope menentukan wilayah program di mana suatu identifier berlaku dan dapat diakses. Bahasa Pascal menggunakan lexical scoping (atau static scoping): scope sebuah identifier ditentukan sepenuhnya oleh posisinya secara tekstual di dalam program, bukan oleh urutan pemanggilan saat runtime. Konsekuensinya, suatu identifier di scope dalam (inner) dapat menyembunyikan identifier dengan nama yang sama di scope luar (name hiding atau shadowing).

Untuk mendukung pencarian identifier lintas scope secara efisien, digunakan struktur display: sebuah array yang memetakan setiap lexical level ke blok yang aktif pada level tersebut. Saat sebuah identifier dicari, kompiler menelusuri display dari level terdalam ke level terluar hingga identifier ditemukan atau dinyatakan tidak terdefinisi.

1.8. Static dan Dynamic Link

Pada mesin stack berbasis blok, setiap kali subprogram dipanggil, sebuah activation record (atau stack frame) dibuat di atas stack untuk menampung konteks eksekusi subprogram tersebut. Activation record minimal terdiri dari tiga slot wajib di bagian

awal, yaitu static link, dynamic link, dan return address, yang diikuti oleh slot untuk parameter dan variabel lokal.

1.8.1.Static Link

Static link adalah penunjuk dari suatu activation record ke activation record dari enclosing scope-nya secara leksikal, yaitu blok yang secara tekstual melingkupi subprogram tersebut di dalam kode sumber. Static link digunakan pada saat runtime untuk mengakses variabel non-lokal: ketika sebuah instruksi perlu mengakses variabel yang berada di scope luar, mesin virtual menelusuri rantai static link sebanyak selisih lexical level antara titik akses dan titik deklarasi variabel tersebut. Karena hubungan ini ditentukan oleh struktur teks program, static link bersifat tetap dan dapat dihitung saat kompilasi.

1.8.2.Dynamic Link

Dynamic link adalah penunjuk dari suatu activation record ke activation record pemanggil langsungnya (caller), tanpa memperhatikan hubungan leksikal. Dynamic link digunakan ketika sebuah subprogram selesai dieksekusi: mesin virtual mengikuti dynamic link untuk memulihkan stack pointer dan kembali ke frame pemanggil. Berbeda dengan static link, dynamic link ditentukan oleh urutan pemanggilan saat runtime sehingga tidak dapat diprediksi pada waktu kompilasi.

1.8.3.Perbedaan Esensial

Kedua mekanisme tautan tersebut melayani tujuan yang secara fundamental berbeda satu sama lain. Static link berkaitan dengan akses data non-lokal: mekanisme ini menjawab pertanyaan mengenai frame mana yang menyimpan suatu variabel non-lokal, di mana jawabannya telah dapat ditentukan pada waktu kompilasi berdasarkan struktur leksikal program. Sebaliknya, dynamic link berkaitan dengan pemulihan kendali alur eksekusi: mekanisme ini menjawab pertanyaan mengenai frame mana yang harus diaktifkan kembali setelah suatu subprogram menyelesaikan eksekusinya, di mana jawabannya baru dapat diketahui pada waktu runtime sesuai dengan urutan pemanggilan yang terjadi. Dengan demikian, kombinasi kedua mekanisme tautan tersebut

memungkinkan bahasa pemrograman yang mendukung subprogram bersarang (nested subprograms), seperti Pascal, untuk mengelola lingkungan eksekusi secara benar dan efisien pada setiap tingkat kedalaman blok.

1.9. Virtual Machine

Virtual machine (VM) adalah sebuah perangkat lunak yang melakukan perilaku sebuah komputer fisik secara penuh, menciptakan lingkungan eksekusi terisolasi yang berjalan di atas perangkat keras nyata. Berbeda dengan program biasa yang langsung berinteraksi dengan sistem operasi dan perangkat keras, VM bertindak sebagai lapisan abstraksi yang menyimulasikan keberadaan prosesor, memori, dan komponen komputasi lainnya secara virtual. Dalam konteks bahasa pemrograman, VM berperan sebagai mesin eksekusi yang bertanggung jawab menjalankan kode perantara (intermediate code) yang dihasilkan oleh compiler atau generator kode.

Pendekatan ini memungkinkan sebuah program untuk bersifat platform-independent, karena program tidak dieksekusi langsung oleh perangkat keras melainkan oleh VM yang memahami satu set instruksi standar. Contoh nyata dari konsep ini adalah Java Virtual Machine (JVM) pada bahasa Java dan CPython pada bahasa Python. Dalam bahasa Arion, interpreter yang dibangun pada Milestone IV pada dasarnya berfungsi sebagai sebuah VM sederhana yang membaca daftar instruksi TAC secara sekuensial, mengelola memori internal, serta mengeksekusi setiap instruksi seolah-olah terdapat prosesor virtual yang memprosesnya.

1.10. Stack Machine

Stack machine adalah salah satu model arsitektur mesin komputasi yang menggunakan struktur data tumpukan (stack) sebagai media utama untuk menyimpan operand dan hasil perhitungan sementara selama proses eksekusi. Dalam model ini, seluruh operasi aritmatika maupun logika dilakukan dengan cara mengambil (pop) nilai-nilai dari puncak stack, memprosesnya, lalu mendorong (push) hasilnya kembali ke puncak stack. Arsitektur ini berbeda dari model register machine yang menyimpan nilai-nilai sementara di dalam register bernama. Keunggulan stack machine terletak pada kesederhanaannya: instruksi tidak perlu secara eksplisit menyebutkan alamat operand

karena operand selalu diambil dari puncak stack secara implisit. Hal ini membuat ukuran instruksi menjadi lebih kecil dan proses decoding instruksi menjadi lebih seragam. Dalam implementasi interpreter bahasa Arion, stack machine digunakan untuk mengevaluasi ekspresi, menyimpan nilai variabel lokal, serta mengelola pemanggilan dan pengembalian fungsi atau prosedur. Setiap operasi seperti penjumlahan, perbandingan, atau assignment diimplementasikan dengan mendorong operand ke stack terlebih dahulu, kemudian menjalankan instruksi OPR yang akan mengambil nilai-nilai tersebut untuk diproses.

1.11. Siklus Eksekusi

Siklus eksekusi (execution cycle) adalah proses berulang yang dijalankan oleh sebuah mesin komputasi, baik fisik maupun virtual, untuk membaca dan menjalankan setiap instruksi program secara berurutan. Siklus ini umumnya terdiri dari tiga tahap utama yang dilakukan secara terus-menerus. Tahap pertama adalah Fetch, yaitu mesin membaca instruksi yang saat ini ditunjuk oleh Instruction Pointer (IP) atau Program Counter (PC), sebuah penunjuk internal yang menyimpan alamat instruksi yang akan dieksekusi berikutnya. Tahap kedua adalah Decode, yaitu mesin menguraikan instruksi yang telah dibaca untuk menentukan jenis operasi apa yang harus dilakukan beserta operand-operand yang terlibat, misalnya apakah instruksi tersebut adalah perintah aritmatika, perintah lompatan, atau perintah muat nilai. Tahap ketiga adalah Execute, yaitu mesin melakukan aksi nyata sesuai dengan hasil decoding, seperti melakukan perhitungan, menyimpan nilai ke memori, atau mengubah nilai IP untuk melakukan lompatan. Setelah tahap Execute selesai, IP secara otomatis bergerak maju ke instruksi berikutnya ($IP = IP + 1$), kecuali instruksi tersebut adalah instruksi lompatan seperti JMP atau JPC yang akan mengubah nilai IP secara langsung ke baris tujuan. Siklus ini terus berulang hingga instruksi RET dijumpai atau seluruh daftar instruksi TAC telah habis dieksekusi.

1.12. LIFO stack frame

LIFO (Last In, First Out) adalah prinsip operasional yang menjadi dasar dari struktur data stack, di mana elemen yang paling terakhir dimasukkan ke dalam tumpukan akan menjadi elemen yang pertama kali dikeluarkan. Analogi sederhananya adalah

tumpukan piring: piring yang diletakkan paling atas adalah piring yang pertama kali diambil. Dalam konteks interpreter dan manajemen memori program, prinsip LIFO diimplementasikan melalui mekanisme Stack Frame. Stack frame adalah sebuah blok memori yang dialokasikan di dalam stack setiap kali program memasuki sebuah prosedur atau fungsi baru. Setiap frame menyimpan informasi yang dibutuhkan selama eksekusi blok tersebut, meliputi nilai variabel lokal, static link yang menunjuk ke frame induk secara leksikal, dynamic link yang menunjuk ke frame pemanggil, serta return address yang menentukan ke instruksi mana program harus kembali setelah blok selesai dieksekusi.

Ketika sebuah fungsi dipanggil, frame barunya di-push ke puncak stack dan VM hanya diperbolehkan membaca serta memodifikasi variabel yang berada di dalam frame aktif tersebut. Begitu fungsi selesai dieksekusi, frame-nya di-pop dari stack sehingga seluruh variabel lokal di dalamnya otomatis musnah, dan VM kembali berfokus pada frame di bawahnya sesuai dengan return address yang tersimpan. Mekanisme inilah yang memungkinkan pengelolaan scope variabel berjalan secara otomatis dan efisien, sekaligus menjelaskan mengapa variabel lokal di dalam satu fungsi tidak dapat diakses oleh fungsi lain.

1.13. Runtime Error Handling

Runtime error merupakan kesalahan yang terjadi ketika program sedang berjalan (*runtime*). *Runtime error* dapat terjadi ketika nilai suatu memori memicu perilaku tidak terduga (seperti merubah tipe data). Hal ini dapat menyebabkan program mengeluarkan perilaku yang tidak terduga hingga program gagal dieksekusi. Untuk mencegah hal ini terjadi, diperlukan *runtime error checking* (pengecekan *runtime error*). *Runtime error checking* dilakukan dengan mendeteksi instruksi TAC yang dikirimkan. Program akan kembali melakukan pengecekan tipe data saat runtime.

1.14. Kerentanan Interpreter

Dalam menjalankan kode, program dapat menemukan kode yang salah, berbahaya, bahkan tidak masuk akal. Ketika interpreter menerjemahkan kode ini, dapat terjadi crash yang mengakibatkan program tidak berjalan sebagaimana mestinya. Untuk itu,

diperlukan mekanisme yang mampu membantu interpreter menangani perilaku tersebut. Dalam program ini, interpreter mendeteksi beberapa kerentanan interpreter seperti kerentanan pada stack, kerentanan memori dan akses, kerentanan alur eksekusi, dan kerentanan aritmatika. Untuk mengatasi kerentanan pada stack, program harus melakukan pengecekan pada perubahan stack (seperti pengecekan tipe data, akses stack, dan lainnya). Kerentanan memori dapat dicegah dengan melakukan pengecekan akses memori (apakah valid atau tidak, misal dengan cek batas). Kerentanan alur eksekusi dapat dicegah dengan melakukan validasi sebelum interpreter mengeksekusi TAC.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Gambaran Umum

Sistem kompiler Arion pada Milestone 4 terdiri atas dua komponen utama pada sisi back-end, yaitu Intermediate Code Generator (ICG) dan Interpreter. ICG menerima masukan berupa Decorated AST yang telah divalidasi oleh front-end pada milestone sebelumnya, kemudian menghasilkan daftar instruksi kode perantara berbasis stack machine. Interpreter selanjutnya membaca daftar instruksi tersebut dan mengeksekusinya menggunakan arsitektur stack machine dengan siklus Fetch-Decode-Execute.

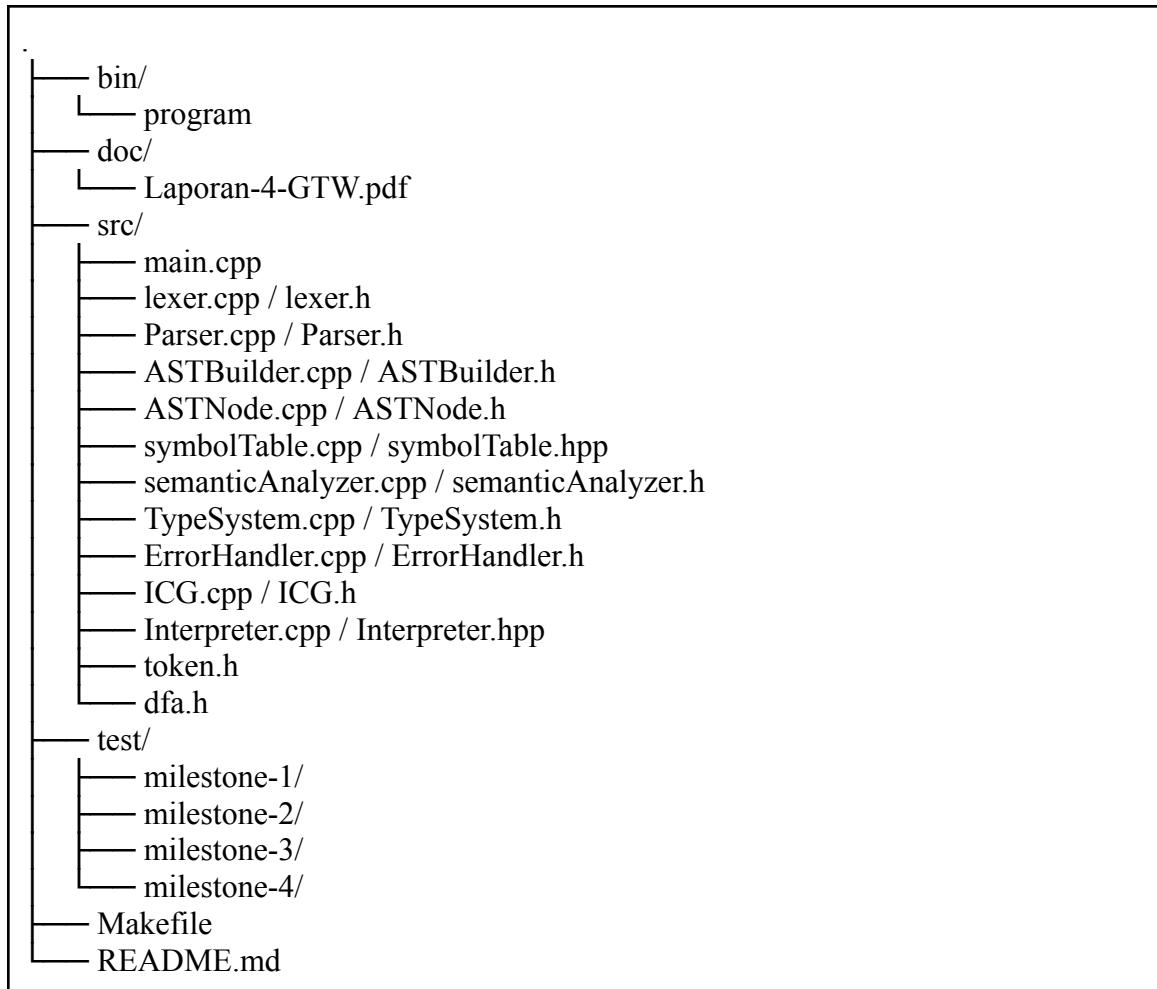
Pipeline eksekusi program Arion secara keseluruhan berjalan dalam satu alur linier tanpa intervensi pengguna di tengah proses. Kode sumber Arion dibaca dari berkas masukan, diproses melalui empat tahap utama, dan menghasilkan keluaran akhir ke terminal serta berkas keluaran. Keempat tahap tersebut adalah analisis leksikal oleh Lexer, analisis sintaksis oleh Parser yang menghasilkan parse tree, analisis semantik oleh SemanticAnalyzer yang menghasilkan Decorated AST, pembangkitan kode perantara oleh ICG, dan eksekusi oleh Interpreter.

Seluruh tahap dijalankan secara sekuensial di dalam fungsi main. Jika terdapat galat sintaksis atau semantik pada tahap front-end, proses dihentikan sebelum memasuki tahap ICG dan Interpreter. Hal ini menjamin bahwa hanya program yang valid secara sintaksis maupun semantik yang akan dieksekusi.

Keluaran program terdiri atas tiga bagian. Bagian pertama adalah parse tree yang dicetak ke terminal. Bagian kedua adalah daftar instruksi kode perantara yang dicetak ke terminal dan disimpan ke berkas keluaran. Bagian ketiga adalah keluaran sesungguhnya dari program Arion, yaitu hasil eksekusi instruksi `writeln` dan `write`, yang dicetak ke terminal sekaligus disimpan ke berkas keluaran melalui mekanisme multi-buffer.

2.2. Struktur File

Repositori proyek menggunakan struktur direktori sebagai berikut.



Berkas-berkas yang ditambahkan secara khusus pada Milestone 4 adalah ICG.cpp, ICG.h, Interpreter.cpp, dan Interpreter.hpp. Berkas ICG.h mendefinisikan struktur data Instruction yang merepresentasikan satu baris kode perantara, enumerasi OpCode untuk jenis instruksi, serta enumerasi OprCode untuk sub-operasi instruksi OPR. Berkas ICG.cpp mengimplementasikan seluruh logika pembangkitan kode perantara. Berkas Interpreter.hpp dan Interpreter.cpp mengimplementasikan mesin virtual yang mengeksekusi daftar instruksi tersebut.

Berkas main.cpp dimodifikasi untuk menyambungkan pipeline front-end yang sudah ada dengan komponen back-end baru. Setelah analisis semantik berhasil tanpa galat, main.cpp membuat objek ICG, memanggil metode generate untuk menghasilkan daftar instruksi, mencetak kode perantara, kemudian membuat objek Interpreter dan memanggil metode run untuk mengeksekusi instruksi tersebut.

2.3. Alur Eksekusi Program

Alur eksekusi program dimulai ketika pengguna menjalankan perintah `./bin/program <nama_file_masukan.txt> <nama_file_keluaran.txt>`. Fungsi `main` membaca seluruh isi berkas masukan ke dalam string, kemudian memproses string tersebut melalui tahap-tahap berikut.

Pada tahap pertama, `Lexer` memecah string kode sumber menjadi daftar token. Pada tahap kedua, `Parser` menerima daftar token dan membangun `parse tree`. Jika terdapat galat sintaksis, pesan galat dicetak dan program berhenti.

Pada tahap ketiga, `ASTBuilder` mengonversi `parse tree` menjadi `AST`, dan `SemanticAnalyzer` melakukan validasi semantik pada `AST` tersebut. Proses ini mengisi tabel simbol dengan informasi setiap identifier serta mendekorasi setiap simpul `AST` dengan informasi tipe data, indeks tabel simbol, dan lexical level. Jika terdapat galat semantik, proses berhenti sebelum memasuki tahap berikutnya.

Pada tahap keempat, `ICG` menerima `Decorated AST` dan tabel simbol sebagai masukan. `ICG` melakukan penelusuran mendalam pada `AST` dan menghasilkan daftar instruksi kode perantara. Proses dimulai dengan instruksi `INT` untuk mengalokasikan memori, dilanjutkan dengan instruksi `JMP` untuk melompati badan subprogram, kemudian pembangkitan kode untuk setiap deklarasi prosedur dan fungsi, dan terakhir pembangkitan kode untuk badan program utama yang ditutup dengan instruksi `RET`.

Pada tahap kelima, `Interpreter` menerima daftar instruksi dari `ICG` beserta tabel string dan tabel bilangan real. `Interpreter` menginisialisasi `stack`, mengatur `Instruction Pointer (IP)` ke baris pertama, dan menjalankan siklus `Fetch-Decode-Execute` hingga instruksi `RET` pada level utama dijumpai. Keluaran program, yaitu hasil dari instruksi `WRT` dan `WRTLN`, dicetak ke terminal dan berkas keluaran secara simultan melalui mekanisme `multi-buffer` yang didefinisikan di `main.cpp`.

2.4. ICG: Struktur Program dan Ekspresi

Implementasi komponen `Intermediate Code Generator` yang menangani struktur program, alokasi memori, evaluasi ekspresi, dan penugasan variabel.

2.4.1. Antarmuka Modul

Seluruh modul ICG dibangun di atas tiga struktur data utama yang didefinisikan di berkas ICG.h. Ketiga struktur ini menjadi kontrak antara ICG dan Interpreter. ICG menghasilkan daftar Instruction, dan Interpreter membaca daftar tersebut untuk dieksekusi.

Struktur pertama adalah enumerasi OpCode yang mendefinisikan jenis instruksi yang dapat dibangkitkan. Enumerasi ini mencakup sembilan instruksi dasar sesuai spesifikasi (LIT, LOD, STO, CAL, INT_OP, JMP, JPC, OPR, RET) serta tiga instruksi tambahan untuk mendukung akses array dan record (LODA, STOA, CKB).

Struktur kedua adalah enumerasi OprCode yang mendefinisikan sub-operasi untuk instruksi OPR. Enumerasi ini mencakup 14 operasi standar sesuai spesifikasi (NEG, ADD, SUB, MUL, DIV, MOD, EQL, NEQ, LSS, GEQ, GTR, LEQ, WRT, WRTLN) serta dua operasi tambahan untuk mendukung pemanggilan subprogram dan akses record (RED dan PUSHBP).

Struktur ketiga adalah Instruction yang merepresentasikan satu baris kode perantara. Setiap instruksi terdiri atas empat bidang: lineNo sebagai nomor baris, opcode sebagai jenis instruksi, level sebagai level relatif atau informasi tipe, dan operand sebagai nilai operan.

```
struct Instruction
{
    int lineNo;
    OpCode opcode;
    int level;
    int operand;
};
```

2.4.2. Struktur Program

Fungsi generate merupakan titik masuk yang dipanggil dari main.cpp. Fungsi ini menginisialisasi ulang daftar instruksi dan penghitung baris, memvalidasi bahwa simpul akar bertipe ProgramNode, kemudian memanggil genProgram.

Fungsi `genProgram` merupakan titik masuk utama pembangkitan kode perantara. Fungsi ini mengatur urutan instruksi untuk keseluruhan program melalui lima langkah.

Langkah pertama adalah memanggil `genINT` untuk mengalokasikan memori pada stack. Langkah kedua adalah menghasilkan instruksi `JMP` dengan target sementara yang berfungsi melompati seluruh badan deklarasi prosedur dan fungsi. Langkah ketiga adalah melakukan iterasi pada daftar deklarasi untuk membangkitkan kode setiap prosedur dan fungsi. Langkah keempat adalah memperbarui target instruksi `JMP` ke baris instruksi pertama dari badan program utama, kemudian membangkitkan kode untuk setiap pernyataan di dalam badan program. Langkah kelima adalah menghasilkan instruksi `RET 0 0` yang menandai akhir eksekusi program.

```
void ICG::genProgram(ProgramNode *node)
{
    genINT(node);
    int jmpPos = currentLine;
    emit(OpCodes::JMP, 0, 0);

    for (auto &decl : node->declarations)
    {
        if (auto *pd = dynamic_cast<ProcDeclNode*>(decl.get()))
            genProcDecl(pd);
        else if (auto *fd = dynamic_cast<FuncDeclNode*>(decl.get()))
            genFuncDecl(fd);
    }

    instructions[jmpPos].operand = currentLine;

    if (node->body)
    {
        auto *compound = dynamic_cast<CompoundStmtNode*>(node->body.get());
        if (compound)
        {
```

```

        for (auto &stmt : compound->statements)
            genStatement(stmt.get());
    }

    emit(OpCode::RET, 0, 0);
}

```

2.4.3. Alokasi Memori

Fungsi `genINT` menghasilkan instruksi INT yang mengalokasikan ruang memori pada stack untuk satu blok program. Ukuran memori dihitung berdasarkan data block table (btab) yang telah diisi oleh analisis semantik. Rumus perhitungannya adalah $\text{frameSize} = 3 + \text{vsze}$, dengan angka 3 merepresentasikan tiga slot wajib pada setiap stack frame (static link, dynamic link, dan return address), sedangkan `vsze` adalah jumlah total slot variabel lokal.

Penggunaan `vsze` dari btab alih-alih menghitung `VarDeclNode` secara manual memberikan keuntungan karena `vsze` sudah memperhitungkan ukuran sebenarnya dari setiap variabel, termasuk array dan record yang membutuhkan lebih dari satu slot.

```

void ICG::genINT(ProgramNode *node)
{
    int ref = st.tab[node->tabIndex].ref;
    int vsze = (ref >= 0 && ref < (int)st.btab.size())
        ? st.btab[ref].vsze
        : 0;
    int memSize = 3 + vsze;
    emit(OpCode::INT_OP, 0, memSize);
}

```

2.4.4. Dispatcher Pernyataan

Fungsi `genStatement` berfungsi sebagai dispatcher yang menentukan jenis simpul pernyataan dan memanggil fungsi pembangkit yang sesuai. Penentuan jenis simpul dilakukan menggunakan `dynamic_cast` terhadap setiap kemungkinan tipe simpul AST.

Jenis pernyataan yang ditangani mencakup penugasan (AssignNode), percabangan (IfNode), perulangan (WhileNode, ForNode, RepeatNode), percabangan majemuk (CaseNode), pemanggilan prosedur (ProcCallNode), deklarasi subprogram yang bersarang (ProcDeclNode, FuncDeclNode), dan blok gabungan (CompoundStmtNode). Untuk deklarasi subprogram yang ditemukan di dalam badan program, dispatcher menghasilkan instruksi JMP untuk melompati badan subprogram dan memperbarui targetnya setelah badan subprogram selesai dibangkitkan.

```
void ICG::genStatement(ASTNode *node)
{
    if (!node)
        return;

    if (auto *n = dynamic_cast<AssignNode *>(node))
        genAssign(n);
    else if (auto *n = dynamic_cast<IfNode *>(node))
        genIf(n);
    else if (auto *n = dynamic_cast<WhileNode *>(node))
        genWhile(n);
    else if (auto *n = dynamic_cast<ForNode *>(node))
        genFor(n);
    else if (auto *n = dynamic_cast<RepeatNode *>(node))
        genRepeat(n);
    else if (auto *n = dynamic_cast<CaseNode *>(node))
        genCase(n);
    else if (auto *n = dynamic_cast<ProcCallNode *>(node))
        genProcCall(n);
    else if (auto *n = dynamic_cast<ProcDeclNode *>(node))
    {
        int jmpPos = currentLine;
        emit(OpCode::JMP, 0, 0);
        genProcDecl(n);
        instructions[jmpPos].operand = currentLine;
    }
    else if (auto *n = dynamic_cast<FuncDeclNode *>(node))
    {
        int jmpPos = currentLine;
```

```

        emit(OpCodes::JMP, 0, 0);
        genFuncDecl(n);
        instructions[jmpPos].operand = currentLine;
    }
    else if (auto *n = dynamic_cast<VarDeclNode *>(node))
    {
        (void)n;
    }
    else if (auto *n = dynamic_cast<CompoundStmtNode
*>(node))
    {
        for (auto &s : n->statements)
            genStatement(s.get());
    }
}

```

2.4.5. Penugasan

Fungsi `genAssign` menangani pernyataan penugasan dengan dua langkah utama. Langkah pertama adalah membangkitkan kode untuk ekspresi di sisi kanan operator `:=` melalui pemanggilan `genExpression`, yang menyimpan hasil evaluasi di puncak stack. Langkah kedua adalah menghasilkan instruksi penyimpanan ke alamat variabel target.

Untuk variabel biasa, alamat diperoleh dari tabel simbol melalui `tabIndex`. Karena variabel pada level 1 atau lebih tinggi disimpan setelah tiga slot wajib stack frame, alamat ditambah 3. Level relatif dihitung sebagai selisih antara `currentLevel` dan level deklarasi variabel, yang menentukan berapa banyak static link yang perlu ditelusuri saat runtime.

Terdapat satu kasus khusus, yaitu ketika target penugasan merupakan nama fungsi itu sendiri. Dalam bahasa Pascal, penugasan ke nama fungsi berarti menyimpan nilai kembalian fungsi tersebut. Pada kasus ini, alamat penyimpanan dihitung sebagai $3 + psze$.

Untuk target berupa akses array atau akses field record, alamat efektif dihitung terlebih dahulu melalui `genArrayAddress` atau `genFieldAccessAddress`,

kemudian instruksi STOA dihasilkan untuk menyimpan nilai ke alamat dinamis yang berada di puncak stack.

```
void ICG::genAssign(AssignNode *node)
{
    genExpression(node->value.get());
    auto *target = node->target.get();

    if (auto *vn = dynamic_cast<VarNode *>(target))
    {
        int addr      = st.tab[vn->tabIndex].adr;
        int varLevel  = st.tab[vn->tabIndex].lev;
        int relLevel  = currentLevel - varLevel;

        if (st.tab[vn->tabIndex].obj == AllowedObj::FUNCTION)
        {
            int ref  = st.tab[vn->tabIndex].ref;
            int psze = (ref > 0 && ref < (int)st.btab.size())
                ? st.btab[ref].psze
                : (int)0;
            emit(OpCode::STO, 0, 3 + psze);
            return;
        }

        if (varLevel >= 1)
            addr += 3;

        emit(OpCode::STO, relLevel, addr);
    }
    else if (auto *an = dynamic_cast<ArrayAccessNode
*>(target))
    {
        genArrayAddress(an);
        emit(OpCode::STOA, 0, 0);
    }
    else if (auto *fn = dynamic_cast<FieldAccessNode
*>(target))
    {
        genFieldAccessAddress(fn);
    }
}
```

```

        emit(OpCodes::STOA, 0, 0);
    }
}

```

2.4.6. Evaluasi Ekspresi

Fungsi `genExpression` berperan sebagai dispatcher yang menentukan jenis simpul ekspresi dan memanggil fungsi pembangkit yang sesuai.

```

void ICG::genExpression(ASTNode *node)
{
    if (!node)
        return;

    if (auto *n = dynamic_cast<NumberNode *>(node))
        genNumber(n);
    else if (auto *n = dynamic_cast<CharNode *>(node))
        genChar(n);
    else if (auto *n = dynamic_cast<StringNode *>(node))
        genString(n);
    else if (auto *n = dynamic_cast<VarNode *>(node))
        genVar(n);
    else if (auto *n = dynamic_cast<BinOpNode *>(node))
        genBinOp(n);
    else if (auto *n = dynamic_cast<UnaryOpNode *>(node))
        genUnaryOp(n);
    else if (auto *n = dynamic_cast<ProcCallNode *>(node))
        genProcCall(n);
    else if (auto *an = dynamic_cast<ArrayAccessNode
*>(node))
    {
        genArrayAddress(an);
        emit(OpCodes::LODA, 0, 0);
    }
    else if (auto *fn = dynamic_cast<FieldAccessNode
*>(node))
    {
        genFieldAccessAddress(fn);
    }
}

```

```

        emit(OpCodes::LODA, 0, 0);
    }
}

```

Untuk literal bilangan bulat, instruksi LIT 0 [nilai] dihasilkan untuk mendorong nilai langsung ke puncak stack. Untuk literal bilangan real, nilai disimpan dalam real pool dengan indeks negatif dan instruksi LIT 0 [indeks] dihasilkan agar interpreter dapat mencari nilai aslinya saat runtime. Pendekatan ini diperlukan karena instruksi LIT hanya menerima operan bertipe integer.

```

void ICG::genNumber(NumberNode *node)
{
    if (node->dtype == DataType::REAL)
    {
        double f = stod(node->rawValue);
        int idx = -1;
        for (auto &[k, v] : realPool)
        {
            if (v == f)
            {
                idx = k;
                break;
            }
        }
        if (idx == -1)
        {
            idx = nextRealIdx--;
            realPool[idx] = f;
        }
        emit(OpCodes::LIT, static_cast<int>(DataType::REAL),
idx);
    }
    else
    {
        int value = stoi(node->rawValue);
        emit(OpCodes::LIT,
static_cast<int>(DataType::INTEGER), value);
    }
}

```


Untuk literal karakter, nilai ASCII karakter digunakan sebagai operan instruksi LIT.

```
void ICG::genChar(CharNode *node)
{
    char c = 0;
    if (!node->rawValue.empty())
    {
        if (node->rawValue.size() >= 3 && node->rawValue[0]
== '\\')
            c = node->rawValue[1];
        else
            c = node->rawValue[0];
    }
    emit(OpCodes::LIT, static_cast<int>(DataType::CHAR),
(int)c);
}
```

Untuk literal string, mekanismenya serupa dengan bilangan real: string disimpan dalam tabel string dengan indeks negatif.

```
void ICG::genString(StringNode *node)
{
    const std::string &s = node->rawValue;

    int idx = -1;
    for (auto &[k, v] : stringTable)
        if (v == s)
        {
            idx = k;
            break;
        }

    if (idx == -1)
    {
        idx = nextStringIdx--;
        stringTable[idx] = s;
    }

    emit(OpCodes::LIT, static_cast<int>(DataType::STRING)
```

```
idx);  
}
```

Untuk variabel, terdapat dua kasus. Jika variabel merupakan konstanta (seperti true atau false), instruksi LIT 0 [nilai] dihasilkan secara langsung karena nilai konstanta sudah tersimpan di bidang adr pada tabel simbol. Jika variabel merupakan variabel biasa, instruksi LOD [relLevel] [alamat] dihasilkan.

```
void ICG::genVar (VarNode *node)  
{  
    int idx = node->tabIndex;  
    if (st.tab[idx].obj == AllowedObj::CONSTANT)  
    {  
        emit(OpCode::LIT, 0,  
static_cast<int>(st.tab[idx].type). adr);  
        return;  
    }  
  
    int addr      = st.tab[idx].adr;  
    int varLevel  = st.tab[idx].lev;  
    int relLevel  = currentLevel - varLevel;  
  
    if (varLevel >= 1)  
        addr += 3;  
  
    emit(OpCode::LOD, relLevel, addr);  
}
```

2.4.7. Operasi Biner dan Uner

Fungsi genBinOp menangani seluruh operasi biner menggunakan pendekatan penelusuran post-order: operan kiri dibangkitkan terlebih dahulu, diikuti operan kanan, kemudian instruksi OPR dengan kode operasi yang sesuai. Pendekatan ini menjamin bahwa kedua operan sudah berada di puncak stack saat instruksi operasi dieksekusi.

```
void ICG::genBinOp (BinOpNode *node)  
{
```

```

genExpression(node->left.get());
genExpression(node->right.get());
string op = node->op;

if (op == "plus")
    emit(OpCode::OPR, 0, (int)OprCode::ADD);
else if (op == "minus")
    emit(OpCode::OPR, 0, (int)OprCode::SUB);
else if (op == "times")
    emit(OpCode::OPR, 0, (int)OprCode::MUL);
else if (op == "slash" || op == "rdiv")
    emit(OpCode::OPR, 0, (int)OprCode::DIV);
else if (op == "div" || op == "idiv")
    emit(OpCode::OPR, 0, (int)OprCode::DIV);
else if (op == "mod" || op == "imod")
    emit(OpCode::OPR, 0, (int)OprCode::MOD);
else if (op == "eq1")
    emit(OpCode::OPR, 0, (int)OprCode::EQL);
else if (op == "neq")
    emit(OpCode::OPR, 0, (int)OprCode::NEQ);
else if (op == "lss")
    emit(OpCode::OPR, 0, (int)OprCode::LSS);
else if (op == "geq")
    emit(OpCode::OPR, 0, (int)OprCode::GEQ);
else if (op == "gtr")
    emit(OpCode::OPR, 0, (int)OprCode::GTR);
else if (op == "leq")
    emit(OpCode::OPR, 0, (int)OprCode::LEQ);
}

```

Pemetaan nama operator menggunakan nama token yang dihasilkan oleh parser, bukan simbol karakter. Berikut tabel pemetaannya.

Token	Kode Opr	Operasi
plus	2	Penjumlahan
minus	3	Pengurangan
times	4	Perkalian
div	5	Pembagian bulat

mod	6	Modulus
eql	7	Sama dengan
neq	8	Tidak sama dengan
lss	9	Kurang dari
geq	10	Lebih dari atau sama dengan
gtr	11	Lebih dari
leq	12	Kurang dari atau sama dengan

Fungsi `genUnaryOp` menangani dua jenis operasi uner. Untuk negasi numerik (operator "-" atau "minus"), instruksi `OPR 0 1 (NEG)` dihasilkan untuk membalik tanda nilai di puncak stack. Untuk negasi logika (operator "not"), nilai operan dibandingkan dengan nol menggunakan instruksi `LIT 0 0` diikuti `OPR 0 7 (EQL)`. Jika nilai operan adalah 0 (false), hasil perbandingan menjadi 1 (true), dan sebaliknya.

2.4.8. Infrastruktur Keluaran

Fungsi `opcodeToString` mengonversi enumerasi `OpCode` menjadi representasi string untuk keperluan pencetakan kode perantara.

```
string ICG::opcodeToString(OpCode op)
{
    switch (op)
    {
        case OpCode::LIT:
            return "LIT";
        case OpCode::LOD:
            return "LOD";
        case OpCode::STO:
            return "STO";
        case OpCode::CAL:
            return "CAL";
        case OpCode::INT_OP:
            return "INT";
        case OpCode::JMP:
```

```

        return "JMP";
    case OpCode::JPC:
        return "JPC";
    case OpCode::OPR:
        return "OPR";
    case OpCode::RET:
        return "RET";
    case OpCode::LODA:
        return "LODA";
    case OpCode::STOA:
        return "STOA";
    case OpCode::CKB:
        return "CKB";
    default:
        return "???";
    }
}

```

Fungsi emit merupakan fungsi pembantu yang digunakan oleh seluruh fungsi pembangkit untuk menambahkan satu instruksi baru ke daftar instruksi. Fungsi ini menerima tiga parameter (opcode, level, operan), membungkusnya ke dalam objek Instruction dengan nomor baris yang sesuai, menambahkannya ke vektor instructions, dan menaikkan penghitung currentLine.

```

void ICG::emit(OpCode op, int level, int operand)
{
    instructions.push_back({currentLine, op, level,
operand});
    currentLine++;
}

```

Fungsi printInstructions mencetak seluruh daftar instruksi ke stream keluaran dengan format empat kolom: nomor baris, nama opcode, level, dan operan. Konversi dari enumerasi OpCode ke string dilakukan oleh fungsi opcodeToString. Fungsi writeToFile merupakan pembungkus yang membuka berkas keluaran dan memanggil printInstructions untuk menuliskan daftar instruksi ke berkas tersebut.

2.5. ICG: Control Flow

2.5.1. Percabangan (genIf)

Fungsi `genIf` bertanggung jawab menghasilkan instruksi TAC untuk pernyataan IF-ELSE. Fungsi ini menerima node `IfNode` yang memiliki tiga komponen utama, yaitu `condition`, `thenBranch`, dan `elseBranch` yang bersifat opsional.

Proses diawali dengan memanggil `genExpression` terhadap node kondisi. Proses ini menghasilkan instruksi-instruksi yang mengevaluasi kondisi dan menempatkan hasilnya berupa nilai 0 atau 1 di puncak stack. Setelah kondisi dievaluasi, instruksi JPC di-emit dengan operan bernilai 0 sebagai placeholder. Posisi instruksi JPC ini disimpan ke variabel `jpcPos` untuk keperluan backpatching. Instruksi JPC akan melompati blok `then` jika kondisi bernilai false.

Selanjutnya `genStatement` dipanggil terhadap node `thenBranch` untuk menghasilkan seluruh instruksi blok `then`. Jika terdapat blok `else`, instruksi JMP di-emit setelah blok `then` selesai dengan operan placeholder. Posisinya disimpan ke variabel `jmpPos`. Instruksi JMP ini berfungsi melewati blok `else` setelah blok `then` selesai dieksekusi. Kemudian operan JPC di-patch dengan nomor baris saat ini yang merupakan awal blok `else`, lalu `genStatement` dipanggil untuk blok `else`. Setelah blok `else` selesai, operan JMP di-patch dengan nomor baris instruksi setelah seluruh blok IF-ELSE. Jika tidak terdapat blok `else`, operan JPC langsung di-patch dengan nomor baris instruksi setelah blok `then`.

```
void ICG::genIf(IfNode *node)
{
    if (!node)
        return;
    genExpression(node->condition.get());

    int jpcPos = currentLine;
    emit(OpCode::JPC, 0, 0); // placeholder

    genStatement(node->thenBranch.get());

    if (node->elseBranch)
```

```

{
    int jmpPos = currentLine;
    emit(OpCodes::JMP, 0, 0); // placeholder
    instructions[jpcPos].operand = currentLine;
    genStatement(node->elseBranch.get());
    instructions[jmpPos].operand = currentLine;
}
else
{
    instructions[jpcPos].operand = currentLine;
}
}

```

2.5.2. Perulangan While (genWhile)

Fungsi `genWhile` bertanggung jawab menghasilkan instruksi TAC untuk pernyataan `WHILE`. Fungsi ini menerima `node WhileNode` yang memiliki dua komponen, yaitu `condition` dan `body`.

Proses diawali dengan mencatat nomor baris instruksi saat ini ke variabel `loopStart` sebelum kondisi di-generate. Nilai ini menjadi target lompatan `JMP` di akhir loop agar eksekusi kembali ke awal untuk mengevaluasi kondisi ulang. Selanjutnya `genExpression` dipanggil untuk menghasilkan instruksi evaluasi kondisi. Instruksi `JPC` di-emit dengan operan placeholder dan posisinya disimpan ke `jpcPos` sebagai pintu keluar loop ketika kondisi bernilai false. Kemudian `genStatement` dipanggil untuk menghasilkan instruksi body loop. Setelah body selesai, instruksi `JMP` di-emit dengan target `loopStart` agar eksekusi kembali ke awal kondisi. Terakhir, operan `JPC` di-patch dengan nomor baris instruksi setelah loop.

Berbeda dari `IF`, `WHILE` adalah pre-condition loop sehingga kondisi selalu dievaluasi sebelum body dieksekusi. Jika kondisi bernilai false sejak awal, `JPC` langsung melompat keluar dan body tidak pernah dieksekusi sama sekali.

```

void ICG::genWhile(WhileNode *node)
{
    if (!node)
        return;
}

```

```

    int loopStart = currentLine;

    genExpression(node->condition.get());
    int jpcPos = currentLine;
    emit(OpCode::JPC, 0, 0); // placeholder

    genStatement(node->body.get());
    emit(OpCode::JMP, 0, loopStart);

    instructions[jpcPos].operand = currentLine;
}

```

2.5.3. Perulangan For (genFor)

Fungsi genFor bertanggung jawab menghasilkan instruksi TAC untuk pernyataan FOR, baik TO maupun DOWNTO. Implementasinya merupakan desugaring dari struktur WHILE dengan penambahan inisialisasi dan increment atau decrement otomatis.

Proses diawali dengan mencari alamat dan level variabel loop di symbol table berdasarkan nama variabel yang tersimpan di node ForNode. Setelah ditemukan, level relatif dihitung dengan rumus $\text{currentLevel} - \text{varLevel}$. Instruksi evaluasi fromExpr di-generate lalu diikuti STO untuk menyimpan nilai awal ke variabel loop.

Nomor baris saat ini disimpan ke loopStart sebagai awal kondisi. Nilai variabel loop di-load dengan LOD, kemudian toExpr di-generate. Untuk FOR TO digunakan OPR LEQ sedangkan untuk FOR DOWNTO digunakan OPR GEQ sebagai kondisi kelanjutan loop. Instruksi JPC di-emit sebagai pintu keluar dengan operan placeholder yang disimpan ke jpcPos.

Setelah JPC, body loop di-generate oleh genStatement. Setelah body, nilai variabel loop di-load kembali, LIT 1 di-push, kemudian OPR ADD untuk TO atau OPR SUB untuk DOWNTO di-emit. Hasilnya disimpan kembali ke variabel loop dengan STO. Instruksi JMP ke loopStart di-emit, dan terakhir JPC di-patch ke instruksi setelah loop.


```

void ICG::genFor(ForNode *node)
{
    if (!node)
        return;

    int varAddr = -1;
    int varLevel = -1;

    for (int i = 0; i < (int)st.tab.size(); i++)
    {
        if (st.tab[i].name == node->varName &&
            st.tab[i].obj == AllowedObj::VARIABLE)
        {
            varAddr = st.tab[i].adr;
            varLevel = st.tab[i].lev;
        }
    }

    if (varAddr < 0)
        throw runtime_error("ICG genFor: variabel loop '" +
node->varName + "' tidak ditemukan di symbol table");

    int relLevel = currentLevel - varLevel;

    genExpression(node->fromExpr.get());
    emit(Opcode::STO, relLevel, varAddr);

    int loopStart = currentLine;
    emit(Opcode::LOD, relLevel, varAddr);
    genExpression(node->toExpr.get());

    if (node->isDownto)
        emit(Opcode::OPR, 0, (int)OprCode::GEQ);
    else
        emit(Opcode::OPR, 0, (int)OprCode::LEQ);

    int jpcPos = currentLine;
    emit(Opcode::JPC, 0, 0);

```

```

        genStatement(node->body.get());

        emit(OpCode::LOD, relLevel, varAddr);
        emit(OpCode::LIT, 0, 1);
        if (node->isDownto)
            emit(OpCode::OPR, 0, (int)OpCode::SUB);
        else
            emit(OpCode::OPR, 0, (int)OpCode::ADD);
        emit(OpCode::STO, relLevel, varAddr);

        emit(OpCode::JMP, 0, loopStart);
        instructions[jpcPos].operand = currentLine;
    }

```

2.5.4. Perulangan Repeat-Until (genRepeat)

Fungsi `genRepeat` bertanggung jawab menghasilkan instruksi TAC untuk pernyataan REPEAT-UNTIL. Berbeda dari WHILE, REPEAT adalah post-condition loop sehingga body selalu dieksekusi minimal satu kali.

Proses diawali dengan mencatat nomor baris saat ini ke `loopStart` sebelum body di-generate. Karena `RepeatNode` menyimpan statements sebagai vector, setiap statement di-generate satu per satu melalui iterasi. Setelah seluruh statement selesai, kondisi until di-generate dengan `genExpression`. Instruksi JPC di-emit dengan target `loopStart`. Karena JPC melompat ketika kondisi bernilai false, loop akan terus berulang selama kondisi belum terpenuhi. Ketika kondisi menjadi true, JPC tidak melompat dan eksekusi berlanjut ke instruksi setelah loop.

Berbeda dari WHILE dan FOR, implementasi REPEAT tidak memerlukan backpatching karena target JPC yaitu `loopStart` sudah diketahui sejak awal sebelum instruksi di-emit.

```

void ICG::genRepeat(RepeatNode *node)
{
    if (!node)
        return;

    int loopStart = currentLine;

```

```

        for (auto &stmt : node->statements)
            genStatement(stmt.get());

        genExpression(node->condition.get());

        emit(OpCode::JPC, 0, loopStart);
    }

```

2.5.5. Percabangan Case (genCase)

Fungsi `genCase` bertanggung jawab menghasilkan instruksi TAC untuk pernyataan CASE-OF. Karena TAC tidak memiliki instruksi switch, CASE diimplementasikan sebagai rangkaian perbandingan berurutan menggunakan pola JPC dan JMP.

Fungsi mengiterasi setiap branch yang tersimpan sebagai pasangan nilai-label dan body. Untuk setiap nilai dalam satu branch, selector dievaluasi ulang dengan `genExpression`, nilai label di-push dengan LIT, kemudian OPR EQL di-emit untuk membandingkan keduanya. Instruksi JPC di-emit untuk melompat ke nilai berikutnya jika tidak cocok, dan JMP di-emit untuk melompat ke body jika cocok. Posisi JMP ini dikumpulkan ke dalam vector `jmpToBodyPositions`.

Setelah semua nilai dalam satu branch dievaluasi, instruksi JMP ke branch berikutnya di-emit untuk kasus di mana tidak ada nilai yang cocok. Body branch kemudian di-generate, diikuti JMP ke after yang posisinya dikumpulkan ke `jmpAfterPositions`. Setelah semua branch selesai di-generate, seluruh posisi JMP ke after di-patch sekaligus dengan nomor baris instruksi setelah blok CASE.

```

void ICG::genCase(CaseNode *node)
{
    if (!node)
        return;

    vector<int> jmpAfterPositions;

    for (auto &branch : node->branches)
    {

```

```

vector<int> jmpToBodyPositions;

for (const string &val : branch.first)
{
    genExpression(node->selector.get());

    try {
        int v = stoi(val);
        emit(Opcode::LIT, 0, v);
    } catch (...) {
        char c = val.empty() ? 0 : val[0];
        emit(Opcode::LIT, 0, (int)c);
    }

    emit(Opcode::OPR, 0, (int)Opcode::EQL);

    int jmpToNextPos = currentLine;
    emit(Opcode::JPC, 0, 0);

    int jmpToBodyPos = currentLine;
    emit(Opcode::JMP, 0, 0);
    jmpToBodyPositions.push_back(jmpToBodyPos);
    instructions[jmpToNextPos].operand = currentLine;
}

int jmpToNextBranch = currentLine;
emit(Opcode::JMP, 0, 0);

int bodyStart = currentLine;
for (int pos : jmpToBodyPositions)
    instructions[pos].operand = bodyStart;

genStatement(branch.second.get());

int jmpAfterPos = currentLine;
emit(Opcode::JMP, 0, 0); // placeholder
jmpAfterPositions.push_back(jmpAfterPos);

```

```

        instructions[jmpToNextBranch].operand = currentLine;
    }

    int afterPos = currentLine;
    for (int pos : jmpAfterPositions)
        instructions[pos].operand = afterPos;
}

```

2.5.6. Penanganan Write dan Writeln (genProcCall)

Fungsi genProcCall bertanggung jawab menghasilkan instruksi TAC untuk pemanggilan prosedur dan fungsi, termasuk prosedur built-in write dan writeln. Untuk write, setiap argumen dievaluasi satu per satu dengan genExpression. Setelah setiap argumen dievaluasi dan nilainya berada di puncak stack, instruksi OPR WRT di-emit untuk mencetak nilai tersebut ke output tanpa newline. Proses ini diulang untuk setiap argumen yang ada.

Untuk writeln, terdapat dua kasus. Jika tidak ada argumen, LIT 0 di-emit sebagai nilai dummy diikuti OPR WRTLN yang mencetak newline saja. Jika terdapat argumen, setiap argumen dievaluasi dengan genExpression. Argumen yang bukan argumen terakhir diikuti OPR WRT, sedangkan argumen terakhir diikuti OPR WRTLN yang mencetak nilai sekaligus newline.

Untuk prosedur dan fungsi user-defined, argumen dievaluasi dan di-push ke stack terlebih dahulu secara berurutan. Level difference dihitung sebagai selisih antara currentLevel dan level definisi prosedur di symbol table. Instruksi CAL di-emit dengan level difference dan nomor baris pertama prosedur sebagai operan.

```

void ICG::genProcCall(ProcCallNode *node)
{
    if (!node)
        return;

    const string &name = node->procName;

    if (name == "write")

```

```

{
    for (auto &arg : node->args)
    {
        genExpression(arg.get());
        emit(OpCode::OPR, 0, (int)OprCode::WRT);
    }
    return;
}

if (name == "writeln")
{
    if (node->args.empty())
    {
        emit(OpCode::LIT, 0, 0);
        emit(OpCode::OPR, 0, (int)OprCode::WRTLN);
    }
    else
    {
        for (int i = 0; i < (int)node->args.size(); i++)
        {
            genExpression(node->args[i].get());
            if (i == (int)node->args.size() - 1)
                emit(OpCode::OPR, 0, (int)OprCode::WRTLN);
            else
                emit(OpCode::OPR, 0, (int)OprCode::WRT);
        }
    }
    return;
}

if (name == "read" || name == "readln")
{
    for (auto &arg : node->args)
    {
        genRecordAddress(arg.get());
        emit(OpCode::OPR, static_cast<int>(arg->dtype),
(int)OprCode::RED);
    }
}

```

```

    }
    return;
}

int procTabIdx = node->tabIndex;
if (procTabIdx < 0)
{
    for (int i = 0; i < (int)st.tab.size(); i++)
    {
        if (st.tab[i].name == name &&
            (st.tab[i].obj == AllowedObj::PROCEDURE ||
             st.tab[i].obj == AllowedObj::FUNCTION))
        {
            procTabIdx = i;
            break;
        }
    }
}

if (procTabIdx < 0)
    throw runtime_error("ICG genProcCall: prosedur '" + name +
        "' tidak ditemukan di symbol table");

for (auto &arg : node->args)
    genExpression(arg.get());

int procDefLevel = st.tab[procTabIdx].lev;
int levelDiff = currentLevel - procDefLevel;

int targetLine = st.tab[procTabIdx].adr;
if (targetLine <= 0)
{
    targetLine = getFuncStartLine(name);
}

emit(OpCode::CAL, levelDiff, targetLine);
}

```

2.6. ICG: Prosedur dan Fungsi

2.6.1. Deklarasi Prosedur (genProcDecl)

Fungsi `genProcDecl` bertanggung jawab menghasilkan seluruh kode antara yang merepresentasikan satu blok prosedur, mulai dari pendefinisian titik masuk (*entry point*), alokasi frame memori, penyalinan parameter, eksekusi tubuh prosedur, hingga instruksi kembali ke pemanggil. Fungsi ini dipanggil satu kali per deklarasi prosedur yang ditemukan dalam AST, baik di tingkat program utama maupun di dalam subprogram bersarang. Setelah `genProcDecl` selesai, symbol table sudah terisi dengan alamat entri prosedur sehingga setiap CAL yang merujuk prosedur ini dapat menggunakan informasi tersebut.

```
void ICG::genProcDecl(ProcDeclNode *node)
{
    int startLine = currentLine;
    st.tab[node->tabIndex].adr = startLine;

    funcStartLine[node->procName] = startLine;
    int ref = st.tab[node->tabIndex].ref;
    int psze = (ref > 0 && ref < (int)st.btab.size()) ?
st.btab[ref].psze : (int)node->params.size();
    int vsze = (ref > 0 && ref < (int)st.btab.size()) ?
st.btab[ref].vsze : 0;
    int frameSize = 3 + psze + vsze;

    emit(OpCode::INT_OP, 0, frameSize);

    for (int p = 0; p < psze; p++)
    {
        emit(OpCode::LOD, 0, -(psze - p)); // load arg dari
bp-(psze-p)
        emit(OpCode::STO, 0, 3 + p); // simpan ke bp+3+p
    }

    currentLevel++;

    if (node->body)
```



```

        genStatement (node->body.get ());
        currentLevel--;

        emit (OpCode::RET, 0, psze);
    }

```

Langkah-langkah implementasi:

a. Pencatatan alamat awal

Sebelum instruksi di tambahkan ke list instruction, nilai `currentLine` saat ini dicatat sebagai alamat entri prosedur. Nilai ini disimpan ke dua tempat: `st.tab[node → tabIndex].adr` agar bisa dibaca `genProcCall` melalui symbol table, dan `funcSatrtLine[name]` sebagai *fallback* cepat berbasis string.

b. Perhitungan ukuran frame

Ukuran frame dihitung dari data block table (btab) yang telah diisi oleh Semantic Analyzer:

- `psze` (parameter size): jumlah slot untuk parameter
- `vsze` (variabel size): jumlah slot untuk variabel lokal
- $\text{frameSize} = 3 + \text{psze} + \text{vsze}$, di mana angka 3 merupakan slot wajib untuk Static Link, Dynamic Link, dan Return Address.

Jika entri btab tidak tersedia (misalnya seperti saat pengujian terisolasi), fallback menggunakan `node→params.size()`.

c. Alokasi Frame

instruksi `INT OP 0 frameSize` ditulis sebagai instruksi pertama tubuh prosedur. Ketika dieksekusi oleh instruksi pertama tubuh prosedur. Ketika dieksekusi oleh Interpreter, instruksi ini memperluas stack sebesar frame size slot, dengan menyiapkan ruang untuk 3 slot wajib (Static Link, Dynamic Link, return Address) ditambah slot untuk parameter dan variabel lokal prosedur.

d. Penyalinan parameter

Setelah `CAL` dieksekusi oleh interpreter, argumen dari *caller* tersimpan di posisi negatif relatif terhadap `bp` baru (`bp-psze`, `bp-(psze-1)`, ...,

bp-1). Loop berikut menyalin setiap argumen ke slot lokal prosedur (bp+3, bp+4, ...):

<pre>LOD 0 -(psze - p) ; muat argumen ke-p dari posisi negatif STO 0 3 + p ; simpan ke slot parameter lokal</pre>
--

e. Eksekusi tubuh

`currentLevel` dinaikkan satu sebelum mengeksekusi tubuh prosedur dan diturunkan setelah selesai. Ini penting agar instruksi LOD/STO yang dibangkitkan untuk variabel di dalam tubuh prosedur menggunakan `relLevel = currentLevel - varLevel` yang benar untuk menentukan jumlah static link hop.

f. Instruksi RET

RET 0 psze mengakhiri prosedur: interpreter akan membersihkan psze argumen dari stack pemanggil dan memulihkan frame sebelumnya.

2.6.2. Deklarasi Fungsi (genFuncDecl)

Fungsi `genFuncDecl` memiliki tanggung jawab yang hampir identik dengan `genProcDecl`, namun dengan satu perbedaan mendasar: fungsi harus mengembalikan sebuah nilai ke pemanggil. Ini berarti frame fungsi perlu mengalokasikan satu slot tambahan untuk menyimpan nilai kembalian tersebut, dan sebelum instruksi RET dieksekusi, nilai dari slot itu harus didorong ke stack agar bisa dibaca oleh pemanggil. `genFuncDecl` juga berkoordinasi dengan `genAssign`, ketika analisis semantik menandai bahwa target sebuah assignment adalah objek bertipe FUNCTION (misalnya `Fakt := 1`), `genAssign` akan menulis hasil ekspresi langsung ke slot nilai kembalian di frame fungsi.

```
void ICG::genFuncDecl(FuncDeclNode *node)
{
    int startLine = currentLine;
```

```

    st.tab[node->tabIndex].adr = startLine;

    funcStartLine[node->funcName] = startLine;
    int ref      = st.tab[node->tabIndex].ref;
    int psze     = (ref > 0 && ref < (int)st.btab.size())
                  ? st.btab[ref].psze :
(int)node->params.size();
    int vsze     = (ref > 0 && ref < (int)st.btab.size())
                  ? st.btab[ref].vsze : 0;

    int frameSize = 3 + psze + vsze + 1;

    emit(OpCode::INT_OP, 0, frameSize);

    for (int p = 0; p < psze; p++)
    {
        emit(OpCode::LOD, 0, -(psze - p));
        emit(OpCode::STO, 0, 3 + p);
    }

    currentLevel++;
    if (node->body)
        genStatement(node->body.get());
    currentLevel--;

    int resultAdr = findReturnVarAddr(node);
    emit(OpCode::LOD, 0, resultAdr);
    emit(OpCode::RET, 1, psze);
}

```

genFucDecl hampir sama dengan genProcDecl, namun berbeda di dua hal yaitu:

a. Slot nilai kembalian

frameSize ditambah satu ($3 + psze + vsze + 1$) untuk mengakomodasi satu slot ekstra tempat nilai kembalian fungsi disimpan. Slot ini nantinya ditulis oleh statement `FunctionName := ekspresi` di dalam

tubuh fungsi (ditangani oleh genAssign yang mendeteksi bahwa target assignment adalah objek FUNCTION).

b. LOD sebelum RET

Sebelum RET, nilai kembalian diambil dari slot-nya di frame dan didorong ke stack:

<pre>LOD 0 resultAdr ; dorong nilai kembalian ke stack RET 1 psze ; kembalikan ke caller, tandai ada return value</pre>
--

RET dengan parameter pertama = 1 menginformasikan interpreter bahwa ada satu nilai di atas stack yang harus diperlakukan sebagai nilai kembalian fungsi

2.6.3. Pemanggilan Subprogram (genProcCall — user-defined)

Fungsi genProcCall bertanggung jawab menghasilkan kode perantara untuk pemanggilan subprogram yang didefinisikan oleh pengguna, yaitu prosedur dan fungsi yang bukan merupakan prosedur bawaan seperti write atau readln. Fungsi ini terdiri dari tiga langkah utama: evaluasi argumen, resolusi alamat target, dan penerbitan instruksi CAL.

```
void ICG::genProcCall(ProcCallNode *node)
{
    if (!node)
        return;

    const string &name = node->procName;

    if (name == "write")
    {
        for (auto &arg : node->args)
        {
            genExpression(arg.get());
            emit(OpCode::OPR, 0, (int)OprCode::WRT);
        }
        return;
    }

    if (name == "writeln")
```

```

{
    if (node->args.empty())
    {
        emit(OpCode::LIT, 0, 0);
        emit(OpCode::OPR, 0, (int)OpCode::WRTLN);
    }
    else
    {
        for (int i = 0; i < (int)node->args.size(); i++)
        {
            genExpression(node->args[i].get());
            if (i == (int)node->args.size() - 1)
                emit(OpCode::OPR, 0, (int)OpCode::WRTLN);
            else
                emit(OpCode::OPR, 0, (int)OpCode::WRT);
        }
    }
    return;
}

if (name == "read" || name == "readln")
{
    for (auto &arg : node->args)
    {
        genRecordAddress(arg.get());
        emit(OpCode::OPR, static_cast<int>(arg->dtype),
(int)OpCode::RED);
    }
    return;
}

int procTabIdx = node->tabIndex;
if (procTabIdx < 0)
{
    for (int i = 0; i < (int)st.tab.size(); i++)
    {
        if (st.tab[i].name == name &&
            (st.tab[i].obj == AllowedObj::PROCEDURE ||

```

```

        st.tab[i].obj == AllowedObj::FUNCTION))
    {
        procTabIdx = i;
        break;
    }
}

if (procTabIdx < 0)
    throw runtime_error("ICG genProcCall: prosedur '" +
name + "' tidak ditemukan di symbol table");

for (auto &arg : node->args)
    genExpression(arg.get());

int procDefLevel = st.tab[procTabIdx].lev;
int levelDiff = currentLevel - procDefLevel;

int targetLine = st.tab[procTabIdx].adr;
if (targetLine <= 0)
{
    targetLine = getFuncStartLine(name);
}

emit(OpCode::CAL, levelDiff, targetLine);
}

```

Langkah-langkah implementasi:

- a. Evaluasi argume: Setiap argumen dievaluasi secara berurutan melalui genExpression, sehingga nilainya terdorong ke puncak stack dari kiri ke kanan sebelum instruksi CAL diterbitkan. Urutan ini penting karena Interpreter mengharapkan argumen tersedia di stack dalam urutan tersebut saat frame baru dialokasikan.

- b. Resolusi alamat target: Alamat instruksi pertama subprogram dibaca dari `st.tab[procTabIdx].adr` yang telah diisi oleh `genProcDecl` atau `genFuncDecl`. Jika nilai tersebut belum valid (≤ 0), digunakan `getFuncStartLine(name)` sebagai fallback berbasis peta string. Selain itu, jika `tabIndex` pada node belum terset (nilai -1), dilakukan pencarian manual ke seluruh `st.tab` untuk menemukan entri dengan nama dan objek yang sesuai.
- c. Perhitungan level relatif: Parameter pertama instruksi CAL adalah selisih `currentLevel - procDefLevel`, yaitu selisih antara level pemanggil dan level tempat subprogram dideklarasikan. Nilai ini digunakan Interpreter untuk menelusuri rantai static link guna memasang static link yang benar pada frame baru. Sebagai contoh, jika program utama (level 0) memanggil prosedur di level 0, $\text{levelDiff} = 1 - 0 = 1$; jika fungsi di level 1 memanggil dirinya sendiri secara rekursif, $\text{levelDiff} = 2 - 1 = 1$.
- d. Instruksi CAL: Instruksi CAL `levelDiff targetLine` memerintahkan Interpreter untuk membuat frame baru, memasang static link dan dynamic link, menyimpan return address, dan mengalihkan Instruction Pointer ke `targetLine`.

2.6.4. Resolusi Alamat Nilai Kembalian (`findReturnVarAddr`)

Fungsi `findReturnVarAddr` bertanggung jawab mencari alamat absolut dalam frame fungsi tempat nilai kembalian disimpan. Ini diperlukan karena `genFuncDecl` perlu mengetahui slot mana yang harus dimuat (LOD) sebelum `RET`, informasi yang tidak tersedia secara langsung dari node AST dan harus dicari dari symbol table. Dalam bahasa Arion, nilai kembalian direpresentasikan sebagai variabel tersembunyi di dalam scope fungsi yang namanya sama dengan nama fungsi itu sendiri. Semantic Analyzer mendaftarkan variabel ini ke `st.tab` saat memproses deklarasi fungsi. Fungsi ini dirancang defensif dengan tiga lapisan pencarian agar tetap berjalan benar bahkan jika konvensi penamaan berbeda atau symbol table tidak lengkap sepenuhnya.

```
int ICG::findReturnVarAddr(FuncDeclNode *node)
```

```

{
    int funcLev = st.tab[node->tabIndex].lev;
    int bodyLev = funcLev + 1;

    // Cari variabel dengan nama = nama fungsi
    for (int i = PredefinedIdx::USER_START; i < (int)st.tab.size();
i++)
    {
        if (st.tab[i].obj == AllowedObj::VARIABLE &&
            st.tab[i].lev == bodyLev &&
            st.tab[i].name == node->funcName)
        {
            return st.tab[i].adr + 3;
        }
    }

    // Fallback 1: cari variabel bernama "result"
    for (int i = PredefinedIdx::USER_START; i < (int)st.tab.size();
i++)
    {
        if (st.tab[i].obj == AllowedObj::VARIABLE &&
            st.tab[i].lev == bodyLev &&
            st.tab[i].name == "result")
        {
            return st.tab[i].adr + 3;
        }
    }

    // Fallback 2: hitung manual dari btab
    int ref = st.tab[node->tabIndex].ref;
    int psze = (ref > 0 && ref < (int)st.btab.size())
                ? st.btab[ref].psze
                : (int)node->params.size();
    return 3 + psze;
}

```

Fungsi ini bertanggung jawab mencari alamat absolut dalam frame fungsi tempat nilai kembalian tersimpan. Dalam bahasa Arion (Pascal-like), nilai kembalian

direpresentasikan sebagai variabel di dalam scope fungsi yang namanya sama dengan nama fungsi itu sendiri. Semantic Analyzer mendaftarkan variabel ini ke `st.tab` saat menganalisis deklarasi fungsi.

Mekanisme pencarian dilakukan dalam tiga lapisan:

1. Pencarian dengan menggunakan nama fungsi: Iterasi `st.tab` mencari entri VARIABLE pada lexical level `bodyLev` ($= \text{level fungsi} + 1$) yang namanya identik dengan nama fungsi. Ini adalah kasus normal.
2. Fallback ke nama “result”: untuk kompatibilitas dengan gaya penulisan Pascal lain yang menggunakan kata kunci `result` sebagai alias nilai kembalian.
3. Fallback hitung manual: jika kedua pencarian gagal (misal saat pengujian unit tanpa symbol table lengkap), alamat dihitung manual sebagai $3 + \text{psze}$, yakni slot tepat setelah parameter.

2.6.5. Akses Array (`genArrayAddress`)

Fungsi `genArrayAddress` bertanggung jawab menghitung alamat absolut sebuah elemen array pada stack dan meletakkan alamat tersebut di puncak stack. Alamat ini kemudian digunakan oleh LODA untuk membaca nilai atau STOA untuk menulis nilai ke elemen tersebut. Pendekatan yang digunakan adalah address arithmetic: mulai dari alamat basis array, lalu tambahkan offset yang dihitung dari indeks dan ukuran elemen.

```
void ICG::genArrayAddress(ArrayAccessNode *an)
{
    genRecordAddress(an->array.get());

    int ref = 0;
    if (auto *vn = dynamic_cast<VarNode *>(an->array.get()))
    {
        ref = st.tab[vn->tabIndex].ref;
    }
    else
    {
        ref = an->array->typeRef;
    }
}
```

```

for (size_t k = 0; k < an->indices.size(); k++)
{
    int low = 0;
    int high = 0;
    int elsz = 1;
    if (ref >= 0 && ref < (int)st.atab.size())
    {
        low = st.atab[ref].low;
        high = st.atab[ref].high;
        elsz = st.atab[ref].elsz;
        ref = st.atab[ref].eref;
    }

    genExpression(an->indices[k].get());

    emit(Opcode::CKB, low, high);

    emit(Opcode::LIT, 0, low);
    emit(Opcode::OPR, 0, (int)OprCode::SUB);

    if (elsz > 1)
    {
        emit(Opcode::LIT, 0, elsz);
        emit(Opcode::OPR, 0, (int)OprCode::MUL);
    }

    emit(Opcode::OPR, 0, (int)OprCode::ADD);
}
}

```

Langkah-langkah implementasi:

- Alamat basis `genRecordAddress` dipanggil untuk mendorong alamat absolut basis array ke stack. Untuk variabel array biasa, ini adalah alamat slot pertama array di frame.
- Dimensi dan batas dari atab Informasi tiap dimensi, batas bawah (low), batas atas (high), dan ukuran per elemen (elsz), dibaca dari `st.atab`

menggunakan referensi ref. Setelah dimensi ke-k diproses, ref diperbarui keeref untuk menangani array multidimensi.

- c. Evaluasi indeks dan pemeriksaan batas Ekspresi indeks dievaluasi melalui genExpression, sehingga nilai indeks berada di puncak stack. Instruksi CKB low high memerintahkan Interpreter untuk memverifikasi bahwa nilai indeks berada dalam rentang [low, high] dan melempar kesalahan jika tidak.

- d. Perhitungan offset

Offset elemen dihitung dengan mengurangi batas bawah dari nilai indeks (indeks – low), kemudian mengalikannya dengan ukuran elemen (elsz) jika elsz > 1. Hasil offset ini ditambahkan ke alamat yang sudah ada di stack melalui instruksi OPR ADD, sehingga puncak stack akhirnya menyimpan alamat absolut elemen yang dituju.

Untuk array satu dimensi bertipe skalar (elsz = 1), instruksi perkalian dihilangkan sebagai optimasi. Untuk array multidimensi atau array of record, loop mengiterasi setiap dimensi secara berurutan, memperbarui ref ke dimensi berikutnya setelah setiap iterasi.

2.6.6. Akses Field Record (genFieldAccessAddress)

Fungsi genFieldAccessAddress bertanggung jawab menghitung alamat absolut sebuah field di dalam record dan meletakkan alamat tersebut di puncak stack. Mekanismenya lebih sederhana dibanding array karena offset setiap field sudah bersifat konstan dan telah dihitung oleh Semantic Analyzer, tidak ada komputasi runtime yang bergantung pada nilai dinamis.

```
void ICG::genFieldAccessAddress(FieldAccessNode *fn)
{
    genRecordAddress(fn->record.get());
    int offset = st.tab[fn->tabIndex].adr;
    if (offset != 0)
    {
        emit(OpCode::LIT, 0, offset);
        emit(OpCode::OPR, 0, (int)OpCode::ADD);
    }
}
```

```
}
```

2.6.7. Fungsi Pembantu (funcStartLine, genRecordAddress)

Field `funcStartLine` dan fungsi `getFuncStartLine` berperan sebagai mekanisme pencarian alamat subprogram alternatif. Masalahnya: pada saat `genProcCall` dieksekusi untuk membangkitkan instruksi `CAL`, ada kemungkinan alamat subprogram belum terekam di `st.tab[]`.`adr`, misalnya jika prosedur dipanggil sebelum deklarasinya selesai diproses. `funcStartLine` menyediakan sumber kebenaran cadangan yang diisi secara proaktif oleh `genProcDecl/genFuncDecl` begitu titik masuk subprogram diketahui, sehingga `genProcCall` selalu bisa mendapatkan alamat target yang valid melalui `getFuncStartLine`.

```
map<string, int> funcStartLine; ;

int ICG::getFuncStartLine(const string& name) const{
    auto it = funcStartLine.find(name);
    if (it == funcStartLine.end()) throw runtime_error("ICG:
subprogram '" + name + "' belum di-generate. " "Pastikan
genProcDecl/genFuncDecl dipanggil sebelum genProcCall.");
    return it->second;
}
```

Field `funcStartLine` merupakan sebuah `map<string, int>` yang memetakan nama subprogram ke nomor baris instruksi pertamanya. Ini diisi setiap kali `genProcDecl` atau `genFuncDecl` dipanggil.

Field ini dibutuhkan oleh `genProcCall` karena ada skenario di mana alamat subprogram belum terdapat di `st.tab[]`.`adr` saat pemanggilan dikompilasi, misalnya jika urutan kompilasi belum memastikan deklarasi selesai sebelum `call-site`. Fungsi `getFuncStartLine` menjadi fallback yang digunakan `genProcCall` jika `st.tab[procTabIdx].adr <= 0`.

Fungsi `genRecordAddress` merupakan fungsi pembantu terpusat yang digunakan oleh `genArrayAddress` dan `genFieldAccessAddress` untuk mendorong alamat absolut basis sebuah objek beralamat (variabel, elemen array, atau field record) ke puncak stack. Dengan memusatkan logika ini di satu tempat, kedua fungsi akses dapat memanggil `genRecordAddress` secara rekursif untuk menangani kasus bersarang tanpa duplikasi kode.

```
void ICG::genRecordAddress(ASTNode *node)
{
    if (auto *vn = dynamic_cast<VarNode *>(node))
    {
        int baseAddr = st.tab[vn->tabIndex].adr;
        int varLevel = st.tab[vn->tabIndex].lev;
        if (varLevel >= 1)
            baseAddr += 3;

        emit(Opcode::OPR, 0, (int)Opcode::PUSHBP);
        int relLevel = currentLevel - varLevel;
        for (int i = 0; i < relLevel; i++)
        {
            emit(Opcode::LODA, 0, 0);
        }
        emit(Opcode::LIT, 0, baseAddr);
        emit(Opcode::OPR, 0, (int)Opcode::ADD);
    }
    else if (auto *an = dynamic_cast<ArrayAccessNode
*>(node))
    {
        genArrayAddress(an);
    }
    else if (auto *fn = dynamic_cast<FieldAccessNode
*>(node))
    {
        genFieldAccessAddress(fn);
    }
    else
    {

```

```

        throw std::runtime_error("ICG: Unsupported base
for address generation");
    }
}

```

Fungsi ini menangani tiga kasus:

a. Variabel biasa (VarNode)

Alamat basis dihitung dari `st.tab[vn->tabIndex].adr`. Untuk variabel lokal ($\text{level} \geq 1$), nilai ini ditambah 3 agar melewati tiga slot wajib di awal frame. Instruksi PUSHBP mendorong nilai base pointer frame aktif ke stack sebagai titik awal perhitungan. Jika variabel berada di scope luar ($\text{level relatif} > 0$), sejumlah instruksi LODA 0 0 diterbitkan untuk menelusuri rantai static link sebanyak `relLevel` kali, sehingga base pointer yang tepat diperoleh. Terakhir, offset variabel ditambahkan melalui LIT dan OPR ADD untuk menghasilkan alamat absolut.

b. Elemen array (ArrayAccessNode)

Jika basis adalah hasil akses array (misalnya untuk array of record), `genArrayAddress` dipanggil secara rekursif. Hasil di puncak stack sudah berupa alamat elemen yang dimaksud sehingga tidak diperlukan langkah tambahan.

c. Field record (FieldAccessNode)

Jika basis adalah hasil akses field (misalnya untuk record bersarang), `genFieldAccessAddress` dipanggil secara rekursif dengan cara yang sama.

2.7. Interpreter dan Stack Machine

2.7.1. Struktur Stack dan Stack Frame

Interpreter diimplementasikan menggunakan struktur `std::vector<StackValue>` sebagai representasi stack utama. Pemilihan vector dibandingkan struktur data lain didasarkan pada kemampuannya untuk mengakses elemen secara acak berdasarkan indeks, yang diperlukan saat Interpreter perlu membaca nilai dari alamat absolut tertentu di dalam stack (misalnya saat operasi LOD dan STO). Struktur data StackValue berisi value dan

tipe data (integer dan DataType). Interpreter memiliki tiga penunjuk utama yang menjadi tulang punggung manajemen memori, yaitu `stackPtr` yang selalu menunjuk ke indeks puncak stack saat ini, `basePtr` yang menunjuk ke pangkal frame aktif yang sedang dieksekusi, dan `pointerCtr` yang berfungsi sebagai Instruction Pointer (IP) untuk melacak instruksi yang sedang dijalankan.

Setiap kali sebuah prosedur atau fungsi dipanggil melalui instruksi `CAL`, Interpreter secara otomatis membangun sebuah Stack Frame baru di puncak stack. Frame tersebut selalu diawali dengan tiga slot khusus yang disusun secara berurutan: slot pertama berisi static link yang menunjuk ke base frame dari lingkup leksikal induknya, slot kedua berisi dynamic link yang menunjuk ke `basePtr` pemanggil, dan slot ketiga berisi return address yang menyimpan nilai `pointerCtr` saat fungsi dipanggil agar eksekusi dapat dilanjutkan kembali setelah fungsi selesai. Setelah ketiga slot tersebut terbentuk, `basePtr` diperbarui ke posisi `stackPtr - 2` sehingga frame baru resmi aktif. Variabel-variabel lokal kemudian dialokasikan di atas ketiga slot tersebut melalui instruksi `INT`.

```
void Interpreter::operateCAL(const Instruction &instr)
{
    int depth = 0;
    int walkBp = basePtr;
    while (walkBp > 0 && depth <= MAX_STACK_DEPTH)
    {
        walkBp = stack[walkBp + 1];
        depth++;
    }
    if (depth >= MAX_STACK_DEPTH)
        throw StackOverflowError("Maximum call depth (" +
std::to_string(MAX_STACK_DEPTH) + ") exceeded");

    int staticLink = base(instr.level, basePtr);
    int dynamicLink = basePtr;
    int returnAddr = pointerCtr;
```

```

        push(staticLink);
        push(dynamicLink);
        push(returnAddr);

        basePtr = stackPtr - 2;
        pointerCtr = instr.operand;
    }

void Interpreter::operateINT(const Instruction &instr)
{
    int m = instr.operand;
    if (m < 0)
        throw RuntimeError("INT operand must be non-negative, got " + std::to_string(m));

    int currentFrameSize = stackPtr - basePtr + 1;
    int toAllocate = m - currentFrameSize;

    for (int i = 0; i < toAllocate; i++)
        push(0);
}

```

2.7.2. Siklus Fetch-Decode-Execute

Siklus eksekusi diimplementasikan sebagai sebuah loop while yang terus berjalan selama nilai `pointerCtr` berada dalam rentang valid indeks instruksi, yaitu antara 0 hingga `code.size() - 1`. Pada setiap iterasi, Interpreter mengambil instruksi pada indeks `pointerCtr` (tahap Fetch), lalu segera menambah `pointerCtr` sebesar satu sebelum instruksi diproses. Pola increment-before-execute ini penting agar instruksi-instruksi lompatan seperti `JMP` dan `JPC` dapat menerima nilai `pointerCtr` setelah instruksi diambil tanpa harus membatalkan increment yang sudah terjadi.

Tahap Decode dilakukan melalui pernyataan switch pada field opcode dari instruksi yang telah diambil. Setiap cabang case mendelegasikan eksekusi ke

fungsi handler yang terpisah seperti `operateLIT`, `operateLOD`, `operateCAL`, dan seterusnya. Pemisahan ini menjaga agar fungsi `run` tetap ringkas dan mudah dibaca. Sebelum siklus utama dimulai, Interpreter juga melakukan validasi awal terhadap seluruh target lompatan yang ada dalam daftar instruksi. Setiap instruksi `JMP` dan `JPC` diperiksa apakah operand-nya menunjuk ke baris yang valid, sehingga kesalahan lompatan dapat terdeteksi lebih awal sebelum program sempat berjalan.

```
void Interpreter::run(const std::vector<Instruction> &code)
{
    if (code.empty())
    {
        return;
    }

    stackPtr = -1;
    basePtr = 0;
    pointerCtr = 0;
    pushCtr = 0;
    popCtr = 0;
    stack.clear();

    for (const auto &ins : code)
    {
        if (ins.opcode == OpCode::JMP || ins.opcode ==
        OpCode::JPC)
        {
            validateJump(ins.operand,
            static_cast<int>(code.size()));
        }
    }

    while (pointerCtr >= 0 && pointerCtr <
    static_cast<int>(code.size()))
    {
        const Instruction &ins = code.at(pointerCtr);

        pointerCtr++;
    }
}
```

```

if (debugMode)
{
    std::cerr << "[PC=" << pointerCtr - 1
                << " BP=" << basePtr
                << " SP=" << stackPtr << "]" << "
                << ICG::opcodeToString(ins.opcode)
                << " " << ins.level
                << " " << ins.operand << "\n";
}

switch (ins.opcode)
{
case OpCode::CAL:
    operateCAL(ins);
    break;
case OpCode::INT_OP:
    operateINT(ins);
    break;
case OpCode::LIT:
    operateLIT(ins);
    break;
case OpCode::LOD:
    operateLOD(ins);
    break;
case OpCode::OPR:
    operateOPR(ins);
    break;
case OpCode::RET:
    operateRET(ins, static_cast<int>(code.size()));
    break;
case OpCode::STO:
    operateSTO(ins);
    break;
case OpCode::JMP:
    operateJMP(ins, static_cast<int>(code.size()));
    break;
case OpCode::JPC:
    operateJPC(ins, static_cast<int>(code.size()));
    break;
case OpCode::LODA:

```

```

    {
        int addr = pop().val;
        push(memLoad(addr));
        break;
    }
    case OpCode::STOA:
    {
        int addr = pop().val;
        StackValue val = pop();
        memStore(addr, val);
        break;
    }
    case OpCode::CKB:
    {
        StackValue indexVal = peek();
        if (indexVal.type != DataType::INTEGER &&
            indexVal.type != DataType::CHAR && indexVal.type !=
            DataType::BOOLEAN && indexVal.type != DataType::SUBRANGE)
        {
            throw RuntimeError("Type mismatch: array index
must be an ordinal type, got " +
SymbolTable::DataTypeToString(indexVal.type));
        }
        int index = indexVal.val;
        int low = ins.level;
        int high = ins.operand;
        if (index < low || index > high)
        {
            throw IndexOutOfBoundsError(index, low, high);
        }
        break;
    }
}
}
}

```

2.7.3. Handler Instruksi (LIT, LOD, STO, INT)

LIT adalah instruksi paling sederhana. Handler `operateLIT` hanya melakukan satu operasi, yaitu memanggil `push(ins.operand, static_cast<DataType>(ins.level))` untuk mendorong nilai literal yang terkandung dalam instruksi ke puncak stack.

LOD diimplementasikan pada `operateLOD` dengan terlebih dahulu menentukan base address dari frame yang dituju menggunakan fungsi pembantu `base(ins.level, basePtr)`. Fungsi `base` menelusuri static link sebanyak level kali untuk menemukan frame yang benar, sehingga Interpreter dapat mengakses variabel dari lingkup leksikal yang lebih luar. Alamat absolut kemudian dihitung sebagai `frameBase + ins.operand`, dan nilai pada alamat tersebut dimuat ke stack melalui `memLoad`.

STO bekerja secara simetris dengan LOD. Handler `operateSTO` terlebih dahulu memanggil `pop()` untuk mengambil nilai dari puncak stack, menghitung alamat tujuan dengan cara yang sama seperti LOD, lalu menyimpan nilai tersebut ke alamat yang dihitung menggunakan `memStore`. Fungsi `memStore` sekaligus memvalidasi bahwa alamat yang dituju berada dalam batas stack yang valid untuk mencegah akses memori ilegal.

INT diimplementasikan pada `operateINT` untuk mengalokasikan ruang variabel lokal dalam frame yang sedang aktif. Handler ini menghitung berapa banyak slot yang masih perlu dialokasikan berdasarkan selisih antara ukuran frame yang diminta (`ins.operand`) dengan ukuran frame saat ini (`stackPtr - basePtr + 1`). Selisih tersebut kemudian dipenuhi dengan mendorong sejumlah nilai nol ke stack, sehingga semua variabel lokal memiliki nilai awal yang terdefinisi.

```
int Interpreter::base(int levels, int b) const
{
    while (levels > 0)
    {
        if (b < 0 || b >= static_cast<int>(stack.size()))
            throw StackCorruptionError("static link out of
bounds during base() traversal");
```

```

        b = stack[b]; // static link: stack[b+0]
        levels--;
    }
    return b;
}

void Interpreter::operateLIT(const Instruction &ins)
{
    push(ins.operand, static_cast<DataType>(ins.level));
}

void Interpreter::operateLOD(const Instruction &ins)
{
    int frameBase = base(ins.level, basePtr);
    int addr = frameBase + ins.operand;
    push(memLoad(addr));
}

void Interpreter::operateSTO(const Instruction &instr)
{
    StackValue val = pop();
    int frameBase = base(instr.level, basePtr);
    int addr = frameBase + instr.operand;
    memStore(addr, val);
}

void Interpreter::operateCAL(const Instruction &instr)
{
    int depth = 0;
    int walkBp = basePtr;
    while (walkBp > 0 && depth <= MAX_STACK_DEPTH)
    {
        walkBp = stack[walkBp + 1].val;
        depth++;
    }
    if (depth >= MAX_STACK_DEPTH)
        throw StackOverflowError("Maximum call depth (" +
std::to_string(MAX_STACK_DEPTH) + ") exceeded");

    int staticLink = base(instr.level, basePtr);

```

```

    int dynamicLink = basePtr;
    int returnAddr = pointerCtr;

    push(staticLink, DataType::INTEGER);
    push(dynamicLink, DataType::INTEGER);
    push(returnAddr, DataType::INTEGER);

    basePtr = stackPtr - 2;
    pointerCtr = instr.operand;
}

void Interpreter::operateINT(const Instruction &instr)
{
    int m = instr.operand;
    if (m < 0)
        throw RuntimeError("INT operand must be non-negative,
got " + std::to_string(m));

    int currentFrameSize = stackPtr - basePtr + 1;
    int toAllocate = m - currentFrameSize;

    for (int i = 0; i < toAllocate; i++)
        push(0, DataType::INTEGER);
}

```

2.7.4. Handler Operasi (OPR: aritmatika, perbandingan, I/O)

Seluruh operasi komputasi diarahkan ke satu handler terpusat, yaitu operateOPR, yang kemudian mendistribusikan eksekusi berdasarkan nilai ins.operand yang di-cast ke enum OprCode.

Untuk operasi aritmatika (NEG, ADD, SUB, MUL, DIV, MOD), setiap operasi melibatkan pop satu atau dua operand dari stack, melakukan kalkulasi, lalu mendorong hasilnya kembali. Operasi-operasi yang berpotensi menghasilkan overflow seperti ADD, SUB, MUL, dan NEG didelegasikan ke fungsi pembantu doAdd, doSub, doMul, dan doNeg yang masing-masing melakukan komputasi menggunakan tipe long long terlebih dahulu, kemudian memeriksa apakah

hasilnya melampaui batas nilai integer yang diizinkan sebelum melakukan cast kembali ke int. Untuk DIV dan MOD, selain pengecekan pembagian oleh nol, juga terdapat penanganan khusus untuk kasus overflow yang terjadi ketika nilai INT_MIN dibagi dengan -1.

Untuk operasi perbandingan (EQL, NEQ, LSS, GEQ, GTR, LEQ), Interpreter mengambil dua operand dari stack, membandingkannya, lalu mendorong hasil berupa nilai integer 1 (true) atau 0 (false) ke puncak stack. Representasi boolean sebagai integer ini konsisten dengan cara instruksi JPC mengonsumsi kondisi lompatan.

Untuk operasi I/O, WRT dan WRTLN sama-sama memanggil fungsi `outputValue` dengan parameter `newline` yang berbeda. Fungsi `outputValue` memiliki logika pengenalan tipe nilai: jika nilai yang di-pop adalah indeks kurang dari atau sama dengan -100000, nilai tersebut diperlakukan sebagai kunci ke dalam `stringTable`; jika nilainya antara -99999 hingga -1, nilai tersebut diperlakukan sebagai kunci ke dalam `realPool`; selain itu nilai dicetak langsung sebagai integer. Operasi RED menangani pembacaan input dari `std::cin` dengan mempertimbangkan tipe data yang diharapkan (integer, real, char, boolean, atau string) berdasarkan field `ins.level`, dan hasil pembacaan disimpan ke alamat memori yang sebelumnya di-pop dari stack.

```
void Interpreter::operateOPR(const Instruction &instr)
{
    OprCode op = static_cast<OprCode>(instr.operand);

    switch (op)
    {
        case OprCode::NEG:
        {
            StackValue a = pop();
            if (a.type != DataType::INTEGER && a.type != DataType::REAL)
            {
```

```

        throw RuntimeError("Type mismatch: operand of NEG must
be numeric, got " + SymbolTable::DataTypeToString(a.type));
    }
    if (a.type == DataType::REAL)
    {
        pushReal(-realPool.at(a.val));
    }
    else
    {
        push(doNeg(a.val), DataType::INTEGER);
    }
    break;
}
case OprCode::ADD:
{
    StackValue b = pop(), a = pop();
    if ((a.type != DataType::INTEGER && a.type !=
DataType::REAL) ||
        (b.type != DataType::INTEGER && b.type !=
DataType::REAL))
    {
        throw RuntimeError("Type mismatch: expected numeric
operands for ADD, got " + SymbolTable::DataTypeToString(a.type) + "
and " + SymbolTable::DataTypeToString(b.type));
    }
    if (a.type == DataType::REAL || b.type == DataType::REAL)
    {
        double valA = (a.type == DataType::REAL) ?
realPool.at(a.val) : a.val;
        double valB = (b.type == DataType::REAL) ?
realPool.at(b.val) : b.val;
        pushReal(valA + valB);
    }
    else
    {
        push(doAdd(a.val, b.val), DataType::INTEGER);
    }
    break;
}
}

```



```

case OprCode::SUB:
{
    StackValue b = pop(), a = pop();
    if ((a.type != DataType::INTEGER && a.type !=
DataType::REAL) ||
        (b.type != DataType::INTEGER && b.type !=
DataType::REAL))
    {
        throw RuntimeError("Type mismatch: expected numeric
operands for SUB, got " + SymbolTable::DataTypeToString(a.type) + "
and " + SymbolTable::DataTypeToString(b.type));
    }
    if (a.type == DataType::REAL || b.type == DataType::REAL)
    {
        double valA = (a.type == DataType::REAL) ?
realPool.at(a.val) : a.val;
        double valB = (b.type == DataType::REAL) ?
realPool.at(b.val) : b.val;
        pushReal(valA - valB);
    }
    else
    {
        push(doSub(a.val, b.val), DataType::INTEGER);
    }
    break;
}
case OprCode::MUL:
{
    StackValue b = pop(), a = pop();
    if ((a.type != DataType::INTEGER && a.type !=
DataType::REAL) ||
        (b.type != DataType::INTEGER && b.type !=
DataType::REAL))
    {
        throw RuntimeError("Type mismatch: expected numeric
operands for MUL, got " + SymbolTable::DataTypeToString(a.type) + "
and " + SymbolTable::DataTypeToString(b.type));
    }
    if (a.type == DataType::REAL || b.type == DataType::REAL)

```

```

        {
            double valA = (a.type == DataType::REAL) ?
realPool.at(a.val) : a.val;
            double valB = (b.type == DataType::REAL) ?
realPool.at(b.val) : b.val;
            pushReal(valA * valB);
        }
        else
        {
            push(doMul(a.val, b.val), DataType::INTEGER);
        }
        break;
    }
    case OprCode::DIV:
    {
        StackValue b = pop(), a = pop();
        if ((a.type != DataType::INTEGER && a.type !=
DataType::REAL) ||
            (b.type != DataType::INTEGER && b.type !=
DataType::REAL))
        {
            throw RuntimeError("Type mismatch: expected numeric
operands for DIV, got " + SymbolTable::DataTypeToString(a.type) + "
and " + SymbolTable::DataTypeToString(b.type));
        }
        double valA = (a.type == DataType::REAL) ?
realPool.at(a.val) : a.val;
        double valB = (b.type == DataType::REAL) ?
realPool.at(b.val) : b.val;
        if (valB == 0.0)
            throw DivisionByZeroError();

        if (a.type == DataType::REAL || b.type == DataType::REAL)
        {
            pushReal(valA / valB);
        }
        else
        {

```

```

        if (b.val == -1 && a.val ==
static_cast<int>(INT_MIN_VAL))
            throw ArithmeticOverflowError("DIV signed overflow
(" + std::to_string(a.val) + " / " + std::to_string(b.val) + ")");
        push(a.val / b.val, DataType::INTEGER);
    }
    break;
}
case OprCode::MOD:
{
    StackValue b = pop(), a = pop();
    if (a.type != DataType::INTEGER || b.type !=
DataType::INTEGER)
    {
        throw RuntimeError("Type mismatch: operands of MOD must
be Integer, got " + SymbolTable::DataTypeToString(a.type) + " and "
+ SymbolTable::DataTypeToString(b.type));
    }
    if (b.val == 0)
        throw DivisionByZeroError();
    if (b.val == -1 && a.val == static_cast<int>(INT_MIN_VAL))
        throw ArithmeticOverflowError("MOD signed overflow (" +
std::to_string(a.val) + " % " + std::to_string(b.val) + ")");
    push(a.val % b.val, DataType::INTEGER);
    break;
}
case OprCode::EQL:
case OprCode::NEQ:
case OprCode::LSS:
case OprCode::GEQ:
case OprCode::GTR:
case OprCode::LEQ:
{
    StackValue b = pop(), a = pop();
    bool res = compareValues(op, a, b);
    push(res ? 1 : 0, DataType::BOOLEAN);
    break;
}
}

```

```

// Output operations
case OprCode::WRT:
{
    int val = pop().val;
    outputValue(val, false);
    break;
}
case OprCode::WRTLN:
{
    int val = pop().val;
    outputValue(val, true);
    break;
}
case OprCode::PUSHBP:
{
    push(basePtr, DataType::INTEGER);
    break;
}
case OprCode::RED:
{
    int addr = pop().val;
    std::string inputStr;
    DataType expectedType = static_cast<DataType>(instr.level);
    bool readSuccess = false;

    if (expectedType == DataType::STRING)
    {
        while (std::cin.peek() == '\n' || std::cin.peek() ==
'\r')
        {
            std::cin.get();
        }
        if (std::getline(std::cin, inputStr))
        {
            readSuccess = true;
        }
    }
    else
    {

```

```

        if (std::cin >> inputStr)
        {
            readSuccess = true;
        }
    }

    if (readSuccess)
    {
        try
        {
            if (expectedType == DataType::REAL)
            {
                size_t parsedChars = 0;
                double doubleVal = std::stod(inputStr,
&parsedChars);

                if (parsedChars < inputStr.size())
                {
                    throw std::invalid_argument("not a pure
double");
                }
                int idx = -1;
                for (auto &[k, v] : realPool)
                {
                    if (v == doubleVal)
                    {
                        idx = k;
                        break;
                    }
                }
                if (idx == -1)
                {
                    int minIdx = -1;
                    for (auto &[k, v] : realPool)
                    {
                        if (k < minIdx) minIdx = k;
                    }
                    idx = minIdx - 1;
                    realPool[idx] = doubleVal;
                }
            }
        }
    }
}

```

```

        memStore(addr, {idx, DataType::REAL});
    }
    else if (expectedType == DataType::INTEGER)
    {
        size_t parsedChars = 0;
        int intVal = std::stoi(inputStr, &parsedChars);
        if (parsedChars < inputStr.size())
        {
            throw std::invalid_argument("not a pure
integer");
        }
        memStore(addr, {intVal, DataType::INTEGER});
    }
    else if (expectedType == DataType::CHAR)
    {
        if (inputStr.size() != 1)
        {
            throw std::invalid_argument("not a char");
        }
        memStore(addr, {static_cast<int>(inputStr[0]),
DataType::CHAR});
    }
    else if (expectedType == DataType::BOOLEAN)
    {
        std::string lowerStr = inputStr;
        for (char &c : lowerStr) c = tolower(c);
        if (lowerStr == "true" || lowerStr == "1")
        {
            memStore(addr, {1, DataType::BOOLEAN});
        }
        else if (lowerStr == "false" || lowerStr == "0")
        {
            memStore(addr, {0, DataType::BOOLEAN});
        }
        else
        {
            throw std::invalid_argument("not a
boolean");
        }
    }

```

```

    }
    else if (expectedType == DataType::STRING)
    {
        int idx = -1;
        for (auto &[k, v] : stringTable)
        {
            if (v == inputStr)
            {
                idx = k;
                break;
            }
        }
        if (idx == -1)
        {
            int minIdx = -100000;
            for (auto &[k, v] : stringTable)
            {
                if (k < minIdx) minIdx = k;
            }
            idx = minIdx - 1;
            stringTable[idx] = inputStr;
        }
        memStore(addr, {idx, DataType::STRING});
    }
    else
    {
        throw std::invalid_argument("unsupported input
type");
    }
}
catch (...)
{
    throw RuntimeError("Invalid input: '" + inputStr +
"'" does not match expected type");
}
}
break;
}

```

```

        default:
            throw RuntimeError("Unknown OPR code: " +
std::to_string(instr.operand));
        }
    }
}

void Interpreter::outputValue(int val, bool newline)
{
    if (val <= -100000 && stringTable.count(val))
    {
        std::string s = stringTable.at(val);
        if (s.size() >= 2 && s.front() == '\\' && s.back() ==
'\')
        {
            s = s.substr(1, s.size() - 2);
        }
        size_t pos = 0;
        while ((pos = s.find("'", pos)) != std::string::npos)
        {
            s.replace(pos, 2, "");
            pos += 1;
        }
        pos = 0;
        while ((pos = s.find("\\n", pos)) !=
std::string::npos)
        {
            s.replace(pos, 2, "\n");
            pos += 1;
        }
        pos = 0;
        while ((pos = s.find("\\t", pos)) !=
std::string::npos)
        {
            s.replace(pos, 2, "\t");
            pos += 1;
        }
        output << s;
    }
    else if (val >= -99999 && val <= -1)
    {
        if (realPool.count(val))

```



```

    {
        double d = realPool.at(val);
        if (d == (int)d)
        {
            output << d << ".0";
        }
        else
        {
            output << d;
        }
    }
    else
    {
        output << val;
    }
}
else
{
    output << val;
}

if (newline)
    output << "\n";
}

```

2.7.5. Handler Control Flow (JMP, JPC, CAL, RET)

JMP diimplementasikan pada operateJMP yang memanggil validateJump terlebih dahulu untuk memastikan target lompatan valid, lalu langsung mengisi pointerCtr dengan nilai ins.operand. Pada iterasi berikutnya, siklus Fetch akan membaca instruksi dari posisi baru tersebut.

JPC pada operateJPC terlebih dahulu memanggil pop() untuk mengambil nilai kondisi dari puncak stack. Jika nilai tersebut adalah 0 (false), lompatan dilakukan dengan mengisi pointerCtr serupa dengan JMP. Jika kondisi bernilai 1 (true), pointerCtr tidak diubah dan eksekusi berlanjut ke instruksi berikutnya secara

sekuensial, mengimplementasikan semantik IF_FALSE GOTO yang dijelaskan dalam teori TAC.

CAL pada operateCAL membangun Stack Frame baru sebelum melompat ke prosedur yang dipanggil. Sebelum frame dibangun, Interpreter memeriksa kedalaman call stack saat ini dengan menelusuri dynamic link secara berulang. Jika kedalaman mencapai batas MAX_STACK_DEPTH, sebuah StackOverflowError dilempar untuk mendeteksi infinite recursion. Jika aman, tiga nilai dikirim ke stack secara berurutan yaitu static link (hasil base(instr.level, basePtr)), dynamic link (nilai basePtr saat ini), dan return address (nilai pointerCtr saat ini). Setelah itu, basePtr diperbarui ke stackPtr - 2 dan pointerCtr diisi dengan instr.operand sebagai titik masuk prosedur.

RET pada operateRET menangani dua skenario: jika basePtr adalah 0, artinya program sedang di scope global dan cukup dengan mengeset pointerCtr ke -1 untuk menghentikan loop eksekusi. Untuk pemanggilan fungsi biasa, handler membaca returnValue dari puncak stack (jika instruksi ini berasal dari fungsi, bukan prosedur, ditandai oleh ins.level == 1), kemudian membaca retAddr dan callerBp dari slot yang tersimpan di dalam frame aktif. Sebelum dikembalikan, kedua nilai tersebut divalidasi: return address yang menunjuk ke luar batas kode atau dynamic link yang nilainya tidak konsisten akan memicu StackSmashingError sebagai penanda korupsi frame. Stack kemudian dikecilkan hingga posisi sebelum frame aktif, basePtr dan pointerCtr dikembalikan ke nilai pemanggil, dan nilai kembalian (jika ada) di-push kembali ke stack.

```
void Interpreter::operateJMP(const Instruction &instr, int
codeSize)
{
    validateJump(instr.operand, codeSize);
    pointerCtr = instr.operand;
}
```

```

void Interpreter::operateJPC(const Instruction &instr, int
codeSize)
{
    StackValue cond = pop();
    if (cond.type != DataType::BOOLEAN && cond.type !=
DataType::INTEGER )
    {
        throw RuntimeError("Type mismatch: condition must be
Boolean or Integer, got " +
SymbolTable::DataTypeToString(cond.type));
    }
    if (cond.val == 0)
    {
        {
            validateJump(instr.operand, codeSize);
            pointerCtr = instr.operand;
        }
    }
}

void Interpreter::operateRET(const Instruction &ins, int
codeSize)
{
    if (basePtr == 0)
    {
        pointerCtr = -1;
        return;
    }

    if (basePtr < 0 || basePtr + 2 > stackPtr)
    {
        throw StackCorruptionError(
            "bp=" + std::to_string(basePtr) + " is invalid
during RET (sp=" + std::to_string(stackPtr) + ")");
    }

    bool isFunction = (ins.level == 1);
    int psize = ins.operand;

```

```

StackValue returnValue;
if (isFunction)
{
    if (stackPtr < 0 || stackPtr >=
static_cast<int>(stack.size()))
        throw StackCorruptionError("No return value
available on stack for function");
    returnValue = stack[stackPtr];
}

int retAddr = stack[basePtr + 2].val;
int callerBp = stack[basePtr + 1].val;

if (retAddr < 0 || retAddr >= codeSize)
{
    throw StackSmashingError("Return address corrupted ("
+ std::to_string(retAddr) + ")");
}

if (callerBp < 0 || callerBp >= basePtr)
{
    throw StackSmashingError("Dynamic link / Caller BP
corrupted (" + std::to_string(callerBp) + ")");
}

int targetSp = basePtr - psze - 1;

while (stackPtr > targetSp)
{
    stack.pop_back();
    stackPtr--;
    popCtr++;
}

basePtr = callerBp;
pointerCtr = retAddr;

if (isFunction)
    push(returnValue);
}

```

2.7.6. Handler Tambahan (LODA, STOA, CKB)

LODA (Load from Absolute Address) menangani akses memori dengan alamat dinamis. Handler ini mengambil satu nilai dari puncak stack, memperlakukannya sebagai alamat absolut, lalu memuat nilai dari alamat tersebut menggunakan `memLoad` dan mendorong hasilnya ke stack. Instruksi ini digunakan untuk mengakses elemen array di mana alamat elemen dihitung secara dinamis pada saat runtime.

STOA (Store to Absolute Address) adalah pasangan dari LODA. Handler ini melakukan dua kali pop: pertama untuk mendapatkan alamat tujuan, dan kedua untuk mendapatkan nilai yang akan disimpan. Nilai tersebut kemudian disimpan ke alamat yang didapat menggunakan `memStore`. Urutan pop yang tepat (alamat lebih dulu dari nilai) mencerminkan urutan push yang dilakukan oleh code generator sebelum instruksi ini dijalankan.

CKB (Check Bounds) adalah instruksi penjaga keamanan akses array. Handler ini membaca nilai di puncak stack menggunakan `peek()` tanpa mengambilnya, lalu membandingkan nilai tersebut dengan batas bawah (`ins.level`) dan batas atas (`ins.operand`) yang sudah dikodekan langsung di dalam instruksi. Jika nilai indeks berada di luar rentang tersebut, sebuah `IndexOutOfBoundsException` dilempar dengan menyertakan informasi indeks yang dicoba diakses beserta rentang yang valid, sehingga pesan kesalahan menjadi informatif bagi pengguna.

```
std::vector<Instruction>
Interpreter::parseFromStream(std::istream &stream)
{
    std::vector<Instruction> code;
    std::string line;

    while (std::getline(stream, line))
    {
        if (line.empty() || line[0] == ';')
```

```

        continue;

    auto semiPos = line.find(';');
    if (semiPos != std::string::npos)
        line = line.substr(0, semiPos);

    std::istringstream iss(line);
    int lineNo;
    std::string mnemonic;
    int level, operand;

    if (!(iss >> lineNo >> mnemonic))
        continue;

    if (mnemonic == "RET")
    {
        code.push_back({lineNo, OpCode::RET, 0, 0});
        continue;
    }

    if (!(iss >> level >> operand))
        continue;

    OpCode op;
    if (mnemonic == "LIT")
        op = OpCode::LIT;
    else if (mnemonic == "LOD")
        op = OpCode::LOD;
    else if (mnemonic == "STO")
        op = OpCode::STO;
    else if (mnemonic == "CAL")
        op = OpCode::CAL;
    else if (mnemonic == "INT")
        op = OpCode::INT_OP;
    else if (mnemonic == "JMP")
        op = OpCode::JMP;
    else if (mnemonic == "JPC")
        op = OpCode::JPC;
    else if (mnemonic == "OPR")
        op = OpCode::OPR;

```

```

        else if (mnemonic == "LODA")
            op = OpCode::LODA;
        else if (mnemonic == "STOA")
            op = OpCode::STOA;
        else if (mnemonic == "CKB")
            op = OpCode::CKB;
        else
            throw RuntimeError("Unknown mnemonic: " +
mnemonic);

        code.push_back({lineNo, op, level, operand});
    }
    return code;
}

```

2.7.7. Tabel String dan Real Pool

Akibat seluruh slot di dalam stack direpresentasikan sebagai int, tipe data yang tidak dapat direpresentasikan secara langsung sebagai integer perlu disimpan secara terpisah. Interpreter menggunakan dua struktur tambahan untuk keperluan ini.

stringTable bertipe `std::map<int, std::string>` digunakan untuk menyimpan nilai bertipe string. Setiap string diberi kunci berupa integer negatif yang nilainya kurang dari atau sama dengan -100000. Nilai yang tersimpan di dalam stack untuk variabel bertipe string hanyalah kunci integer ini, bukan string-nya secara langsung. Ketika `outputValue` mendeteksi nilai di rentang tersebut dan tipe data yang diterima sebagai string, ia mencari string yang sesuai di `stringTable` dan menampilkannya. Tabel ini juga menangani proses unescaping, yaitu mengkonversi pasangan tanda kutip tunggal " menjadi satu karakter kutip ', serta mengkonversi literal `\n` dan `\t` menjadi karakter newline dan tab yang sesungguhnya.

realPool bertipe `std::map<int, double>` bekerja dengan prinsip yang sama untuk tipe data bilangan real. Nilai bertipe `double` disimpan di pool ini dengan kunci integer negatif di rentang -99999 hingga -1. Saat mencetak nilai dari rentang ini, `outputValue` mencari nilai `double` yang bersesuaian di `realPool`. Jika nilai `double` tersebut secara matematis sama dengan nilai integer-nya (misalnya 5.0), maka ia dicetak dengan format 5.0 untuk mempertahankan representasi tipe real yang eksplisit, bukan hanya 5.

```
void Interpreter::outputValue(int val, bool newline)
{
    if (val <= -100000 && stringTable.count(val))
    {
        std::string s = stringTable.at(val);
        if (s.size() >= 2 && s.front() == '\\' && s.back() ==
'\\')
        {
            s = s.substr(1, s.size() - 2);
        }
        size_t pos = 0;
        while ((pos = s.find("'", pos)) != std::string::npos)
        {
            s.replace(pos, 2, "");
            pos += 1;
        }
        pos = 0;
        while ((pos = s.find("\\n", pos)) !=
std::string::npos)
        {
            s.replace(pos, 2, "\\n");
            pos += 1;
        }
        pos = 0;
        while ((pos = s.find("\\t", pos)) !=
std::string::npos)
        {
            s.replace(pos, 2, "\\t");
            pos += 1;
        }
    }
```



```

        output << s;
    }
    else if (val >= -99999 && val <= -1)
    {
        if (realPool.count(val))
        {
            double d = realPool.at(val);
            if (d == (int)d)
            {
                output << d << ".0";
            }
            else
            {
                output << d;
            }
        }
        else
        {
            output << val;
        }
    }
    else
    {
        output << val;
    }

    if (newline)
        output << "\n";
}

```

2.8. Penanganan Galat

2.8.1. Galat Pembagian dengan Nol

Runtime error division by zero didefinisikan pada interpreter.cpp sebagai berikut,

```

struct DivisionByZeroError : public RuntimeError
{
    explicit DivisionByZeroError()
        : RuntimeError("Division by zero") {}
};

```

Kasus ini dapat terjadi ketika dilakukan operasi pembagian dengan pembagi bernilai 0. Untuk itu, error ini ditangani pada fungsi `Interpreter::operateOPR` saat melakukan DIV atau MOD. Program akan melakukan pengecekan pada pembagi dari operasi DIV dan MOD. Jika pembagi bernilai 0, program akan melemparkan `DivisionByZeroError` dengan snippet sebagai berikut,

Case DIV

```
double valA = (a.type == DataType::REAL) ?  
realPool.at(a.val) : a.val;  
double valB = (b.type == DataType::REAL) ?  
realPool.at(b.val) : b.val;  
if (valB == 0.0)  
    throw DivisionByZeroError();
```

Case MOD

```
StackValue b = pop(), a = pop();  
...  
if (b.val == 0)  
    throw DivisionByZeroError();
```

2.8.2. Galat Target Lompatan Tidak Valid

Runtime error division by zero didefinisikan pada `interpreter.cpp` sebagai berikut,

```
struct InvalidJumpError : public RuntimeError  
{  
    explicit InvalidJumpError(int target)  
        : RuntimeError("Invalid jump target: line " +  
std::to_string(target)) {}  
};
```

2.8.3. Galat Tipe Tidak Sesuai

Runtime error type mismatch didefinisikan pada `interpreter.cpp` menggunakan runtime error sebagai berikut,

```
struct RuntimeError : public std::runtime_error
```

```
{
    explicit RuntimeError(const std::string &msg)
        : std::runtime_error("[RuntimeError] " + msg) {}
};
```

Kasus ini terjadi ketika terdapat tipe data yang tidak sesuai saat runtime. Ditangani dengan melakukan pengecekan tipe data saat runtime. Berikut adalah beberapa kasus yang dihandle oleh program,

Interpreter::compareValues

```
if ((a.type == DataType::STRING || b.type == DataType::STRING) &&
a.type != b.type)
{
    throw RuntimeError("Type mismatch: cannot compare String and
" + SymbolTable::DataTypeToString(b.type == DataType::STRING ?
a.type : b.type));
}
if ((a.type == DataType::BOOLEAN || b.type == DataType::BOOLEAN)
&& a.type != b.type)
{
    throw RuntimeError("Type mismatch: cannot compare Boolean
and " + SymbolTable::DataTypeToString(b.type == DataType::BOOLEAN ?
a.type : b.type));
}
```

Interpreter::operateJPC

```
StackValue cond = pop();
if (cond.type != DataType::BOOLEAN && cond.type !=
DataType::INTEGER)
{
    throw RuntimeError("Type mismatch: condition must be Boolean
or Integer, got " + SymbolTable::DataTypeToString(cond.type));
}
```

2.8.4. Stack Overflow dan Underflow

Runtime error stack overflow dan underflow didefinisikan pada interpreter.cpp sebagai berikut,

```
struct StackOverflowError : public RuntimeError
```

```
{
    explicit StackOverflowError(const std::string &msg = "Stack
overflow: maximum frame depth exceeded")
        : RuntimeError(msg) {}
};

struct StackUnderflowError : public RuntimeError
{
    explicit StackUnderflowError(const std::string &msg = "Stack
underflow: pop on empty stack")
        : RuntimeError(msg) {}
};
```

Kasus ini terjadi ketika program ingin melakukan overwrite melebihi batas maksimum yang ditentukan untuk stack (bukan batas saat ini, tetapi batas maksimum yang dialokasi). Kasus ini dicek pada Interpreter::operateCAL, Interpreter::peek, dan Interpreter::pop dengan snippet sebagai berikut, operateCAL

```
if (depth >= MAX_STACK_DEPTH)
    throw StackOverflowError("Maximum call depth (" +
std::to_string(MAX_STACK_DEPTH) + ") exceeded");
```

Peek & pop

```
if (stackPtr < 0 || stack.empty())
{
    throw StackUnderflowError();
}
```

2.8.5. Akses di Luar Batas

Runtime error outofbound didefinisikan pada interpreter.cpp sebagai berikut,

```
struct OutOfBoundsError : public RuntimeError
{
    explicit OutOfBoundsError(int addr, int base, int limit)
        : RuntimeError("Memory access out of bounds: address " +
std::to_string(addr) + " (frame base=" + std::to_string(base) + ",
limit=" + std::to_string(limit) + ")") {}
```

```
};

struct IndexOutOfBoundsError : public RuntimeError
{
    explicit IndexOutOfBoundsError(int index, int low, int high)
        : RuntimeError("Index out of bounds: index " +
std::to_string(index) + " is not in range [" + std::to_string(low) +
".." + std::to_string(high) + "]") {}
};
```

Kasus ini terjadi ketika program ingin mengakses elemen yang berada di luar stack atau memori yang telah didefinisi sekarang (atau akses ke tempat yang tidak seharusnya).

Interpreter::run case OpCode::CKB

```
if (index < low || index > high)
{
    throw IndexOutOfBoundsError(index, low, high);
}
```

Interpreter::memLoad

```
if (addr < 0 || addr >= static_cast<int>(stack.size()))
    throw OutOfBoundsError(addr, basePtr,
static_cast<int>(stack.size()));
```

Interpreter::memStore

```
if (addr < 0 || addr >= static_cast<int>(stack.size()))
    throw OutOfBoundsError(addr, basePtr,
static_cast<int>(stack.size()));
```

2.8.6. Arithmetic Overflow

Runtime error aritmatika didefinisikan pada interpreter.cpp sebagai berikut,

```
struct ArithmeticOverflowError : public RuntimeError
{
    explicit ArithmeticOverflowError(const std::string &op)
        : RuntimeError("Arithmetic overflow in operation: " + op) {}
};
```

Kasus ini terjadi ketika operasi aritmatika menyebabkan nilai dari integer melewati batas yang ditentukan (baik batas atas maupun batas bawah). Kasus ini dicek pada operasi ADD, SUB, MUL, NEG, DIV, dan MOD. Berikut adalah snippet umum yang digunakan untuk melakukan pengecekan pada operasi aritmatika.

```
    if (r > INT_MAX_VAL || r < INT_MIN_VAL)
        throw ArithmeticOverflowError("<CASE> (" + std::to_string(a)
+ " + " + " + std::to_string(b) + ")");
```

BAB III

PENGUJIAN

3.1. Metode Pengujian

Pengujian dilakukan dengan menjalankan program compiler Arion menggunakan command:

```
./bin/program <input.txt> <output.txt>
```

Program akan mencetak Parse Tree, Intermediate Code, dan Symbol Table ke terminal, serta menjalankan program melalui interpreter dan mencetak hasilnya ke layar dan file output.

3.2. Hasil Pengujian

3.2.1. Kasus Uji 1

- Kasus: Terdapat kasus *overwrite* data di stack yang mengakibatkan tipe data berubah
- Tujuan: Menguji perbandingan variabel dengan tipe data berbeda
- Input:

```
program TypeMismatch;
type
  Corruptor == record
    arr: array[1.. -4] of integer;
    fakeString: integer
  end;
var
  idx: string;
  c: Corruptor;
begin
  idx := 'hello';
  c.fakeString := 999;
  if idx == 'hello' then
    writeln('Matches!')
  end.
```

- Output:

```

=== Intermediate Code ===
0 INT 0 4
1 JMP 0 2
2 LIT 4 -100000
3 STO 0 3
4 LIT 0 999
5 OPR 0 15
6 LIT 0 7
7 OPR 0 2
8 LIT 0 -4
9 OPR 0 2
10 STOA 0 0
11 LOD 0 3
12 LIT 4 -100000
13 OPR 0 7
14 JPC 0 17
15 LIT 4 -100001
16 OPR 4 14
17 RET 0 0

=== Output Program ===
[RuntimeException] Type mismatch: cannot compare String and Integer

```

- Analisis:

Program berhasil mendeteksi *runtime error type mismatch*. Namun, program belum dapat melakukan pencegahan pada deklarasi array berindeks negatif (arr: array[1.. -4] of integer;) yang mengakibatkan variabel idx di-*overwrite*.

3.2.2. Kasus Uji 2

- Kasus: Terdapat fungsi input readln untuk menerima nilai yang akan dimasukkan oleh user ke variabel. Pada kasus ini menguji fungsi input dan write pada tipe data array serta record.
- Tujuan: Kasus ini bertujuan untuk memastikan nilai yang di-output pada array dan record sesuai dengan input dari user
- Input:

```

program TestReadWriteRobustness;

type

```



```

Item == record
  id: integer;
  price: real
end;

var
  laptop: Item;
  ratings: array[1..3] of real;
  status: boolean;
  userMsg: string;

begin

  writeln('--- TEST 1: Escape & Primitif ---');

  { Menguji teks kirimanmu: literal backslash + escaped
quote }
  writeln('hai\\');

  { Menguji cetak boolean langsung }
  writeln(true);
  writeln(false);

  { Menguji teks dengan petik ganda di dalam string }
  writeln('Mengecek fitur "READLN" dan "WRITELN"
Arion');

  writeln('--- TEST 2: Array of Real ---');
  writeln('Masukkan nilai rating ke-1 (contoh: 4.5):');
  readln(ratings[1]);

  writeln('Masukkan nilai rating ke-2 (contoh: 3.14):');
  readln(ratings[2]);

  writeln('Masukkan nilai rating ke-3 (contoh: 5.0):');
  readln(ratings[3]);

  writeln('Menampilkan isi Array Ratings (Wajib
Desimal/Real):');
  writeln(ratings[1]);
  writeln(ratings[2]);
  writeln(ratings[3]);

```

```

writeln('--- TEST 3: Record Field ---');
writeln('Masukkan ID Barang (Integer, contoh: 99):');
readln(laptop.id);

writeln('Masukkan Harga Barang (Real, contoh:
1250.75):');
readln(laptop.price);

writeln('Menampilkan isi Record Laptop:');
write('ID Laptop: ');
writeln(laptop.id);
write('Harga Laptop ($): ');
writeln(laptop.price);

writeln('--- TEST 4: String Echo ---');
writeln('Ketikkan satu kata/kalimat tanpa petik:');
readln(userMsg);
write('Pesan yang Anda ketik: ');
writeln(userMsg);

writeln('--- ALL TESTS COMPLETED ---');
end.

```

- Output:

```

=== Intermediate Code ===
0 INT 0 17
1 JMP 0 2
2 LIT 0 -100000
3 OPR 0 14
4 LIT 0 -100001
5 OPR 0 14
6 LIT 0 1
7 OPR 0 14
8 LIT 0 0
9 OPR 0 14
10 LIT 0 -100002
11 OPR 0 14
12 LIT 0 -100003
13 OPR 0 14
14 LIT 0 -100004
15 OPR 0 14
16 OPR 0 15

```

17 LIT 0 6
18 OPR 0 2
19 LIT 0 1
20 CKB 1 3
21 LIT 0 1
22 OPR 0 3
23 LIT 0 2
24 OPR 0 4
25 OPR 0 2
26 OPR 1 16
27 LIT 0 -100005
28 OPR 0 14
29 OPR 0 15
30 LIT 0 6
31 OPR 0 2
32 LIT 0 2
33 CKB 1 3
34 LIT 0 1
35 OPR 0 3
36 LIT 0 2
37 OPR 0 4
38 OPR 0 2
39 OPR 1 16
40 LIT 0 -100006
41 OPR 0 14
42 OPR 0 15
43 LIT 0 6
44 OPR 0 2
45 LIT 0 3
46 CKB 1 3
47 LIT 0 1
48 OPR 0 3
49 LIT 0 2
50 OPR 0 4
51 OPR 0 2
52 OPR 1 16
53 LIT 0 -100007
54 OPR 0 14
55 OPR 0 15
56 LIT 0 6
57 OPR 0 2
58 LIT 0 1
59 CKB 1 3
60 LIT 0 1
61 OPR 0 3
62 LIT 0 2

63 OPR 0 4
64 OPR 0 2
65 LODA 0 0
66 OPR 0 14
67 OPR 0 15
68 LIT 0 6
69 OPR 0 2
70 LIT 0 2
71 CKB 1 3
72 LIT 0 1
73 OPR 0 3
74 LIT 0 2
75 OPR 0 4
76 OPR 0 2
77 LODA 0 0
78 OPR 0 14
79 OPR 0 15
80 LIT 0 6
81 OPR 0 2
82 LIT 0 3
83 CKB 1 3
84 LIT 0 1
85 OPR 0 3
86 LIT 0 2
87 OPR 0 4
88 OPR 0 2
89 LODA 0 0
90 OPR 0 14
91 LIT 0 -100008
92 OPR 0 14
93 LIT 0 -100009
94 OPR 0 14
95 OPR 0 15
96 LIT 0 3
97 OPR 0 2
98 OPR 0 16
99 LIT 0 -100010
100 OPR 0 14
101 OPR 0 15
102 LIT 0 3
103 OPR 0 2
104 LIT 0 1
105 OPR 0 2
106 OPR 1 16
107 LIT 0 -100011
108 OPR 0 14

```
109 LIT 0 -100012
110 OPR 0 13
111 OPR 0 15
112 LIT 0 3
113 OPR 0 2
114 LODA 0 0
115 OPR 0 14
116 LIT 0 -100013
117 OPR 0 13
118 OPR 0 15
119 LIT 0 3
120 OPR 0 2
121 LIT 0 1
122 OPR 0 2
123 LODA 0 0
124 OPR 0 14
125 LIT 0 -100014
126 OPR 0 14
127 LIT 0 -100015
128 OPR 0 14
129 OPR 0 15
130 LIT 0 13
131 OPR 0 2
132 OPR 4 16
133 LIT 0 -100016
134 OPR 0 13
135 LOD 0 13
136 OPR 0 14
137 LIT 0 -100017
138 OPR 0 14
139 RET 0 0
```

=== Output Program ===

--- TEST 1: Escape & Primitif ---

hai\\'

1

0

Mengecek fitur 'READLN' dan 'WRITELN' Arion

--- TEST 2: Array of Real ---

Masukkan nilai rating ke-1 (contoh: 4.5):

Masukkan nilai rating ke-2 (contoh: 3.14):

Masukkan nilai rating ke-3 (contoh: 5.0):

Menampilkan isi Array Ratings (Wajib Desimal/Real):

4.2

3.11111

3.0

```
--- TEST 3: Record Field ---  
Masukkan ID Barang (Integer, contoh: 99):  
Masukkan Harga Barang (Real, contoh: 1250.75):  
Menampilkan isi Record Laptop:  
ID Laptop: 3  
Harga Laptop ($): 44.2222  
--- TEST 4: String Echo ---  
Ketikkan satu kata/kalimat tanpa petik:  
Pesan yang Anda ketik: Hello, Arion!  
--- ALL TESTS COMPLETED ---
```

- Analisis: Program berhasil melalui uji kasus ini dengan memberikan output yang sesuai dengan input yang diberikan. Hasil output dapat dilihat pada text yang di-highlight warna kuning serta untuk fungsi input sudah handle ketika tipe data yang di-input berbeda dengan yang diharapkan program. Handle berupa pesan runtime error yang menyebabkan program berhenti. Terdapat batasan juga ketika nilai rating ke-3 dimasukkan 2.9999 output program dibulatkan ke atas menjadi 3.0.

3.2.3. Kasus Uji 3

- Kasus: Mendeklarasikan integer di nilai batas dan menaikkan nilainya melebihi batas
- Tujuan: menguji ketika terjadi *integer overflow*
- Input:

```
program TestOverflow;  
var  
  x: integer;  
begin  
  x := 2147483647;  
  x := x + 1;  
  writeln(x)  
end.
```

- Output:

```
=== Intermediate Code ===  
0 INT 0 4
```

```

1 JMP 0 2
2 LIT 0 2147483647
3 STO 0 3
4 LOD 0 3
5 LIT 0 1
6 OPR 0 2
7 STO 0 3
8 LOD 0 3
9 OPR 0 14
10 RET 0 0

```

=== Output Program ===

[RuntimeError] Arithmetic overflow in operation: ADD (2147483647 + 1)

- Analisis:

Program telah berhasil mendeteksi ketika terjadi *integer overflow* dan mendeteksi penyebabnya.

3.2.4. Kasus Uji 4

- Kasus: Ketika terjadi pembagian dengan 0
- Tujuan: menguji *runtime error division by zero*
- Input:

```

program TestDivByZero;
var
  x, y: integer;
begin
  x := 10;
  y := x div 0;
  writeln(y)
end.

```

- Output:

```

=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 10
3 STO 0 3

```

```
4 LOD 0 3
5 LIT 0 0
6 OPR 0 5
7 STO 0 4
8 LOD 0 4
9 OPR 0 14
10 RET 0 0
```

```
=== Output Program ===
[RuntimeError] Division by zero
```

- Analisis:

Program berhasil mendeteksi pembagian dengan nol.

3.2.5. Kasus Uji 5

- Kasus: melakukan *overwrite return address*
- Tujuan: Menguji *runtime error* ketika *return address invalid*.
- Input:

```
program TestStackSmashing;
type
  Corruptor == record
    arr: array[1.. -3] of integer;
    staticLink: integer;
    dynamicLink: integer;
    returnAddr: integer
  end;

procedure p;
var
  c: Corruptor;
begin
  c.returnAddr := 999999;
end;

begin
  p();
end.
```

- Output:

```
=== Intermediate Code ===
```



```

0 INT 0 3
1 JMP 0 11
2 INT 0 3
3 LIT 0 999999
4 OPR 0 15
5 LIT 0 3
6 OPR 0 2
7 LIT 0 -1
8 OPR 0 2
9 STOA 0 0
10 RET 0 0
11 CAL 0 2
12 RET 0 0

=== Output Program ===
[RuntimeError] Return address corrupted (999999)

```

- Analisis:

Program berhasil mendeteksi ketika *return address corrupted*.

3.2.6. Kasus Uji 6

- Kasus: Prosedur sederhana dengan satu parameter integer, dipanggil dua kali dengan argumen berbeda (variabel dan literal)
- Tujuan: Menguji bahwa `genProcDecl` menghasilkan frame yang benar, penyalinan parameter dari posisi negatif ke slot lokal berjalan benar, dan instruksi `CAL/RET` bekerja untuk prosedur tanpa nilai kembalian.
- Input:

```

program TestProsedur;

var x : integer;

procedure Sapa(n : integer);
begin
  writeln(n)
end;

begin
  x := 42;
  Sapa(x);

```

```
Sapa(7)
end.
```

- Output:

```
=== Intermediate Code ===
0 INT 0 4
1 JMP 0 8
2 INT 0 4
3 LOD 0 -1
4 STO 0 3
5 LOD 0 3
6 OPR 0 14
7 RET 0 1
8 LIT 0 42
9 STO 0 3
10 LOD 0 3
11 CAL 0 2
12 LIT 0 7
13 CAL 0 2
14 RET 0 0

=== Output Program ===
42
7
```

- Analisis:

Program berhasil dieksekusi dan menghasilkan output yang benar. Baris 0 mengalokasikan frame program utama (3 slot wajib + 1 variabel x). Baris 1 adalah JMP 0 8 yang melompati deklarasi prosedur langsung ke body utama, ini adalah hasil koordinasi JMP-backpatch antara genProgram dan genProcDecl. Deklarasi prosedur Sapa dimulai di baris 2: INT 0 4 mengalokasikan frame prosedur (3 wajib + 1 parameter n), kemudian LOD 0 -1 memuat argumen dari posisi negatif dan STO 0 3 menyimpannya ke slot parameter. Baris 7 RET 0 1 mengakhiri prosedur dan membersihkan 1 argumen dari stack. Pemanggilan Sapa(x) di baris 11 dan Sapa(7) di baris 13 keduanya menggunakan CAL 0 2, menunjukkan alamat entri prosedur (baris 2) berhasil direkam oleh funcStartLine.

3.2.7. Kasus Uji 7

- Kasus: Fungsi rekursif faktorial dengan satu parameter, dipanggil dari program utama dan memanggil dirinya sendiri.
- Tujuan: Menguji bahwa `genFuncDecl` menghasilkan slot nilai kembalian, instruksi LOD sebelum RET 1 mendorong nilai kembalian ke stack, dan rekursi melalui CAL 1 bekerja dengan benar.
- Input:

```
program TestFaktorial;

var hasil : integer;

function Fakt(n : integer) : integer;
begin
  if n <= 1 then
    Fakt := 1
  else
    Fakt := n * Fakt(n - 1)
  end;

begin
  hasil := Fakt(5);
  writeln(hasil)
end.
```

- Output:

```
=== Intermediate Code ===
0 INT 0 4
1 JMP 0 21
2 INT 0 5
3 LOD 0 -1
4 STO 0 3
5 LOD 0 3
6 LIT 0 1
7 OPR 0 12
8 JPC 0 12
9 LIT 0 1
10 STO 0 4
11 JMP 0 19
12 LOD 0 3
```

```
13 LOD 0 3
14 LIT 0 1
15 OPR 0 3
16 CAL 1 2
17 OPR 0 4
18 STO 0 4
19 LOD 0 4
20 RET 1 1
21 LIT 0 5
22 CAL 0 2
23 STO 0 3
24 LOD 0 3
25 OPR 0 14
26 RET 0 0
```

```
=== Output Program ===
120
```

- Analisis:

Program berhasil menghasilkan output $120 = 5!$ yang benar. Frame fungsi Fakt dialokasikan dengan INT 0 5 (3 wajib + 1 parameter $n + 1$ slot nilai kembalian Fakt). Baris 19–20 menunjukkan pola khas fungsi: LOD 0 4 memuat nilai kembalian dari slotnya, kemudian RET 1 1 mengembalikan kontrol ke pemanggil sambil mendorong nilai tersebut ke stack. Pemanggilan rekursif di baris 16 menggunakan CAL 1 2, level 1 karena fungsi berada satu level di atas scope-nya sendiri. Assignment Fakt := 1 dan Fakt := $n * \text{Fakt}(n-1)$ di baris 10 dan 18 masing-masing menyimpan ke slot $\text{bp}+4$ sesuai findReturnVarAddr.

3.2.8. Kasus Uji 8

- Kasus: Fungsi dengan dua parameter dipanggil dari dalam prosedur yang juga memiliki dua parameter dan satu variabel lokal.
- Tujuan: Menguji bahwa fungsi dan prosedur bersarang saling memanggil dengan benar, resolusi alamat parameter di frame masing-masing subprogram tepat, dan variabel lokal prosedur tidak bertabrakan dengan slot parameter.
- Input:

```

program TestMultiParam;

function Maks(a : integer; b : integer) : integer;
begin
    if a > b then
        Maks := a
    else
        Maks := b
    end;

procedure CetakMaks(x : integer; y : integer);
var m : integer;
begin
    m := Maks(x, y);
    writeln(m)
end;

begin
    CetakMaks(10, 20);
    CetakMaks(99, 3);
    CetakMaks(7, 7)
end.`

```

- Output:

```

=== Intermediate Code ===
0 INT 0 3
1 JMP 0 30
2 INT 0 6
3 LOD 0 -2
4 STO 0 3
5 LOD 0 -1
6 STO 0 4
7 LOD 0 3
8 LOD 0 4
9 OPR 0 11
10 JPC 0 14
11 LOD 0 3
12 STO 0 5
13 JMP 0 16
14 LOD 0 4
15 STO 0 5
16 LOD 0 5
17 RET 1 2
18 INT 0 6

```

```

19 LOD 0 -2
20 STO 0 3
21 LOD 0 -1
22 STO 0 4
23 LOD 0 3
24 LOD 0 4
25 CAL 1 2
26 STO 0 3
27 LOD 0 3
28 OPR 0 14
29 RET 0 2
30 LIT 0 10
31 LIT 0 20
32 CAL 0 18
33 LIT 0 99
34 LIT 0 3
35 CAL 0 18
36 LIT 0 7
37 LIT 0 7
38 CAL 0 18
39 RET 0 0

=== Output Program ===
20
99
7

```

- Analisis:

Program berhasil menghasilkan output yang benar untuk ketiga pemanggilan. Frame fungsi Maks berukuran 6 (3 wajib + 2 parameter a, b + 1 retval). Baris 3–6 menunjukkan penyalinan dua parameter: LOD 0 -2 dan LOD 0 -1 mengambil argumen dari posisi negatif, disimpan ke bp+3 dan bp+4. Frame prosedur CetakMaks juga berukuran 6 (3 wajib + 2 parameter x, y + 1 variabel lokal m). Pemanggilan Maks(x, y) di baris 25 menggunakan CAL 1 2 , level 1 karena Maks didefinisikan satu level di atas pemanggil CetakMaks. Nilai kembalian Maks yang mendarat di stack langsung disimpan ke m melalui STO 0 3 di baris 26, menunjukkan integrasi antara nilai kembalian fungsi dan variabel lokal prosedur berjalan benar.

3.2.9. Kasus Uji 9

- **Kasus:**

Fungsi dengan dua parameter dan satu variabel lokal, dipanggil tiga kali dengan input berbeda termasuk edge case nilai nol dan parameter kedua nol.

- **Tujuan:**

Menguji bahwa frameSize dihitung benar untuk kombinasi parameter dan variabel lokal, resolusi alamat variabel lokal dari btab.vsize akurat, dan fungsi menghasilkan output benar untuk berbagai kombinasi input.

- **Input:**

```
program TestVarLokal;

var hasil : integer;

function KuadratPlus(n : integer; tambah : integer) : integer;
var temp : integer;
begin
    temp := n * n;
    KuadratPlus := temp + tambah
end;

begin
    hasil := KuadratPlus(3, 1);
    writeln(hasil);
    hasil := KuadratPlus(0, 5);
    writeln(hasil);
    hasil := KuadratPlus(10, 0);
    writeln(hasil)
end.
```

- **Output:**

```
=== Intermediate Code ===
0 INT 0 4
1 JMP 0 17
2 INT 0 7
3 LOD 0 -2
4 STO 0 3
5 LOD 0 -1
```

```

6 STO 0 4
7 LOD 0 3
8 LOD 0 3
9 OPR 0 4
10 STO 0 3
11 LOD 0 3
12 LOD 0 4
13 OPR 0 2
14 STO 0 5
15 LOD 0 5
16 RET 1 2
17 LIT 0 3
18 LIT 0 1
19 CAL 0 2
20 STO 0 3
21 LOD 0 3
22 OPR 0 14
23 LIT 0 0
24 LIT 0 5
25 CAL 0 2
26 STO 0 3
27 LOD 0 3
28 OPR 0 14
29 LIT 0 10
30 LIT 0 0
31 CAL 0 2
32 STO 0 3
33 LOD 0 3
34 OPR 0 14
35 RET 0 0

=== Output Program ===
10
5
100

```

- Analisis:** Program menghasilkan output yang benar: $3^2+1=10$, $0^2+5=5$, $10^2+0=100$. Frame fungsi KuadratPlus berukuran $3 + 2$ (psze) + 1 (vsze) + 1 (retval) = 7 . Variabel lokal temp menempati slot bp+5, tepat setelah dua slot parameter di bp+3 dan bp+4. Kasus KuadratPlus(0, 5) menguji bahwa perkalian $0 * 0 = 0$ lalu ditambah 5 menghasilkan 5 tanpa overflow atau

kesalahan resolusi alamat. Kasus KuadratPlus(10, 0) menguji bahwa penambahan nilai 0 di akhir tidak merusak nilai kembalian. Ketiganya membuktikan bahwa findReturnVarAddr berhasil menemukan slot KuadratPlus di symbol table dan LOD sebelum RET 1 2 membaca nilai yang benar.

3.2.10. Kasus Uji 10

- Kasus: Fungsi rekursif Fibonacci dengan kondisi bercabang tiga ($n \leq 0$, $n \leq 1$, rekursi ganda), dipanggil berulang dalam loop while untuk nilai 0 sampai 7.
- Tujuan: Menguji ketahanan rekursi dalam (dua pemanggilan CAL per frame), kebenaran nilai kembalian fungsi pada kondisi dasar dan rekursif, serta interaksi antara genFuncDecl, genWhile, dan genProcCall dalam satu program.
- Input:

```
program TestFibonacci;

var i : integer;
var r : integer;

function Fib(n : integer) : integer;
begin
  if n <= 0 then
    Fib := 0
  else if n <= 1 then
    Fib := 1
  else
    Fib := Fib(n - 1) + Fib(n - 2)
  end;
end;

begin
  i := 0;
  while i <= 7 do
    begin
      r := Fib(i);
      writeln(r);
      i := i + 1
    end
  end.
end.
```

- Output:

=== Intermediate Code ===

```
0 INT 0 5
1 JMP 0 31
2 INT 0 5
3 LOD 0 -1
4 STO 0 3
5 LOD 0 3
6 LIT 0 0
7 OPR 0 12
8 JPC 0 12
9 LIT 0 0
10 STO 0 4
11 JMP 0 29
12 LOD 0 3
13 LIT 0 1
14 OPR 0 12
15 JPC 0 19
16 LIT 0 1
17 STO 0 4
18 JMP 0 29
19 LOD 0 3
20 LIT 0 1
21 OPR 0 3
22 CAL 1 2
23 LOD 0 3
24 LIT 0 2
25 OPR 0 3
26 CAL 1 2
27 OPR 0 2
28 STO 0 4
29 LOD 0 4
30 RET 1 1
31 LIT 0 0
32 STO 0 3
33 LOD 0 3
34 LIT 0 7
35 OPR 0 12
36 JPC 0 47
37 LOD 0 3
38 CAL 0 2
39 STO 0 4
40 LOD 0 4
41 OPR 0 14
42 LOD 0 3
```

```
43 LIT 0 1
44 OPR 0 2
45 STO 0 3
46 JMP 0 33
47 RET 0 0
```

=== Output Program ===

```
0
1
1
2
3
5
8
13
```

- Analisis: Program menghasilkan 8 bilangan Fibonacci pertama (F(0) hingga F(7)) dengan benar. Deret 0 1 1 2 3 5 8 13 sesuai definisi matematika Fibonacci. Kasus $n=0$ menguji kondisi dasar pertama (Fib := 0), kasus $n=1$ menguji kondisi dasar kedua (Fib := 1), dan kasus $n \geq 2$ menguji rekursi ganda di mana satu frame Fib memanggil dua CAL secara berurutan. Slot nilai kembalian di tiap frame tetap terisolasi dengan benar meskipun stack berkembang dalam sehingga findReturnVarAddr dan mekanisme LOD/RET 1 bekerja benar di setiap level rekursi. Loop while $i \leq 7$ membuktikan integrasi antara genFuncDecl dan genWhile tidak menimbulkan konflik state pada currentLevel.

3.2.11. Kasus Uji 11

- Kasus: Ekspresi aritmatika lengkap
- Tujuan: Memverifikasi bahwa seluruh operasi aritmatika (penjumlahan, pengurangan, perkalian, pembagian bulat, dan modulus) menghasilkan instruksi OPR dengan kode operasi yang benar serta menghasilkan nilai yang tepat saat dieksekusi.
- Input:

```
program TestAritmatika;
var a, b, c, d, e, f: integer;
begin
```

```

a := 10;
b := 3;
c := a + b;
d := a - b;
e := a * b;
f := a div b;
writeln(c);
writeln(d);
writeln(e);
writeln(f);
writeln(a mod b)
end.

```

- Output:

```

=== Output Program ===
13
7
30
3
1

```

- Analisis:

Setiap operator aritmatika berhasil dipetakan ke kode OPR yang sesuai. Instruksi INT 0 9 dihasilkan di awal (3 slot wajib + 6 variabel). Setiap penugasan membangkitkan urutan instruksi yang benar: evaluasi ekspresi sisi kanan terlebih dahulu melalui LOD dan LIT, diikuti OPR untuk operasi, dan STO untuk menyimpan hasil. Keluaran program sesuai dengan hasil perhitungan manual, yang menunjukkan bahwa pemetaan operator dan urutan evaluasi post-order berjalan dengan benar.

3.2.12. Kasus Uji 12

- Kasus: Operasi uner
- Tujuan: Memverifikasi bahwa operator uner negasi (-) menghasilkan instruksi OPR 0 1 (NEG) dan operator not menghasilkan urutan instruksi LIT 0 0 diikuti OPR 0 7 (EQL) untuk membalik nilai boolean.
- Input:

```

program TestUner;
var a: integer; b: boolean;
begin

```

```

a := -5;
writeln(a);
a := -(3 + 2);
writeln(a);
b := not true;
writeln(b);
b := not false;
writeln(b)
end.

```

- Output:

```

=== Output Program ===
-5
-5
0
1

```

- Analisis: Untuk $a := -5$, instruksi yang dihasilkan adalah LIT 0 5 diikuti OPR 0 1 (NEG) yang membalik tanda nilai di puncak stack menjadi -5. Untuk $a := -(3 + 2)$, ekspresi dalam kurung dievaluasi terlebih dahulu (LIT 0 3, LIT 0 2, OPR 0 2), kemudian hasilnya dinegasikan dengan OPR 0 1. Untuk $b := \text{not true}$, konstanta true dimuat dengan LIT 0 1, kemudian urutan LIT 0 0 dan OPR 0 7 (EQL) membandingkan 1 dengan 0, menghasilkan 0 (false). Sebaliknya, not false membandingkan 0 dengan 0, menghasilkan 1 (true). Keluaran program sesuai dengan hasil yang diharapkan.

3.2.13. Kasus Uji 13

- Kasus: Ekspresi bersarang kompleks
- Tujuan: Memverifikasi bahwa evaluasi post-order pada ekspresi bersarang yang melibatkan tanda kurung dan beberapa operator menghasilkan urutan instruksi yang benar serta nilai akhir yang tepat.
- Input:

```

program TestEkspresiKompleks;
var x, y, z: integer;
begin
  x := (3 + 4) * (10 - 2);
  y := x div 7 + x mod 7;
  z := (y * y) - (x + 1);

```

```
writeln(x);  
writeln(y);  
writeln(z)  
end.
```

- Output:

```
=== Output Program ===  
56  
8  
7
```

- Analisis: Ekspresi $(3 + 4) * (10 - 2)$ dievaluasi melalui penelusuran post-order. Cabang kiri menghasilkan instruksi LIT 0 3, LIT 0 4, dan OPR 0 2 (ADD) dengan hasil 7. Cabang kanan menghasilkan instruksi LIT 0 10, LIT 0 2, dan OPR 0 3 (SUB) dengan hasil 8. Selanjutnya, instruksi OPR 0 4 (MUL) mengalikan kedua hasil tersebut menjadi 56. Ekspresi $x \div 7 + x \bmod 7$ menguji bahwa dua operasi berbeda pada variabel yang sama menghasilkan hasil sementara yang tersimpan dengan benar di stack, yaitu $56 \div 7 = 8$ dan $56 \bmod 7 = 0$ yang dijumlahkan menjadi 8. Ekspresi $(y * y) - (x + 1)$ menguji operasi bertingkat lebih lanjut, yaitu $8 * 8 = 64$ dan $56 + 1 = 57$ yang dikurangkan menjadi 7. Seluruh keluaran sesuai dengan hasil perhitungan manual sehingga membuktikan bahwa urutan evaluasi post-order dan pemetaan operator berjalan dengan benar.

3.2.14. Kasus Uji 14

- Kasus: Banyak variabel
- Tujuan: Memverifikasi bahwa instruksi INT mengalokasikan memori dengan ukuran yang benar dan setiap variabel mendapat alamat yang unik serta tidak tumpang tindih.
- Input:

```
program TestBanyakVar;  
var a, b, c, d, e, f, g, h: integer;  
begin  
  a := 1;  
  b := 2;  
  c := 3;  
  d := 4;
```

```

e := a + b;
f := c + d;
g := e * f;
h := g - a;
writeln(h)
end.

```

- Output:

```

=== Output Program ===
20

```

- Analisis: Dengan delapan variabel, instruksi INT 0 11 dihasilkan (3 slot wajib ditambah 8 variabel). Setiap variabel mendapat alamat berurutan mulai dari 3, yaitu setelah tiga slot wajib stack frame: a di alamat 3, b di alamat 4, hingga h di alamat 10. Perhitungan berlangsung secara berantai: $e = 1 + 2 = 3$, $f = 3 + 4 = 7$, $g = 3 * 7 = 21$, dan $h = 21 - 1 = 20$. Apabila terdapat alamat yang tumpang tindih, nilai variabel sebelumnya akan tertimpa dan hasil akhir tidak akan bernilai 20. Keluaran yang benar membuktikan bahwa alokasi memori melalui instruksi INT serta resolusi alamat melalui instruksi LOD dan STO berjalan dengan tepat untuk jumlah variabel yang banyak.

3.2.15. Kasus Uji 15

- Kasus: Tipe data campuran
- Tujuan: Memverifikasi bahwa literal karakter dikonversi ke nilai ASCII melalui instruksi LIT dan literal bilangan real disimpan dalam real pool dengan indeks negatif serta ditampilkan dengan benar oleh interpreter.
- Input:

```

program TestTipeCampuran;
var c: char; r: real;
begin
  c := 'Z';
  writeln(c);
  r := 3.14;
  writeln(r)
end.

```

- Output:

```
=== Output Program ===
```

```
90
```

```
3.14
```

- Analisis: Untuk penugasan $c := 'Z'$, instruksi LIT 0 90 dihasilkan karena nilai ASCII karakter Z adalah 90. Saat dicetak oleh instruksi WRTLN, interpreter mendeteksi bahwa tipe data variabel tersebut adalah char dan menampilkan karakter Z, bukan angka 90. Untuk penugasan $r := 3.14$, ICG menyimpan nilai 3,14 ke dalam real pool dan menghasilkan instruksi LIT 0 -1 dengan indeks negatif pertama. Saat interpreter mengeksekusi instruksi tersebut, indeks negatif menandakan bahwa nilai sebenarnya perlu dicari di real pool. Saat dicetak, interpreter menampilkan nilai desimal asli. Keluaran yang benar membuktikan bahwa mekanisme real pool untuk bilangan real dan konversi karakter ke nilai ASCII berjalan dengan tepat.

3.2.16. Kasus Uji 16

- Kasus: Program mencakup seluruh fitur yang diimplementasikan oleh Anggota 2, meliputi percabangan IF-ELSE, perulangan WHILE, FOR TO, FOR DOWNT0, REPEAT-UNTIL, CASE-OF, pemanggilan prosedur dan fungsi user-defined, serta instruksi write dan writeln. Program juga menguji fungsi faktorial yang menggabungkan FOR dengan pemanggilan fungsi rekursif satu tingkat.
- Tujuan: Kasus ini bertujuan untuk memastikan seluruh komponen ICG yang diimplementasikan Anggota 2 dapat bekerja secara terpadu dalam satu program tanpa konflik antar komponen.
- Input:

```
program TC16;  
var  
  x : integer;  
  y : integer;  
  i : integer;  
  hasil : integer;
```



```

procedure cetakDua(a : integer; b : integer);
begin
    write(a);
    writeln(b)
end;

function tambah(a : integer; b : integer) : integer;
begin
    tambah := a + b
end;

function faktorial(n : integer) : integer;
var
    f : integer;
begin
    f := 1;
    for i := 1 to n do
        begin
            f := f * i
        end;
    faktorial := f
end;

begin
    { === IF-ELSE === }
    x := 10;
    if x > 5 then
        writeln(x);

    x := 3;
    if x > 5 then
        writeln(1)
    else
        writeln(0);

    { === WHILE === }
    x := 1;
    y := 0;
    while x <= 5 do
        begin
            y := y + x;
            x := x + 1
        end;
    writeln(y);

    { === FOR TO === }

```

```

hasil := 0;
for i := 1 to 5 do
begin
    hasil := hasil + i
end;
writeln(hasil);

{ === FOR DOWNT0 === }
hasil := 0;
for i := 5 downto 1 do
begin
    hasil := hasil + i
end;
writeln(hasil);

{ === REPEAT === }
x := 1;
y := 0;
repeat
    y := y + x;
    x := x + 1
until x == 6;
writeln(y);

{ === CASE === }
x := 2;
case x of
    1: writeln(100);
    2: writeln(200);
    3: writeln(300)
end;

{ === WRITE / WRITELN === }
write(4);
write(2);
writeln(0);

{ === PROCEDURE CALL === }
cetakDua(7, 8);

{ === FUNCTION CALL === }
hasil := tambah(10, 32);
writeln(hasil);

{ === FUNCTION DENGAN LOOP === }
hasil := faktorial(5);

```

```
writeln(hasil)
end.
```

- Output:

```
=== Output Program ===
10
0
15
15
15
15
200
420
78
42
120
```

- Analisis: Program berhasil melalui uji kasus ini. Setiap baris output sesuai dengan hasil yang diharapkan. Nilai 15 muncul tiga kali berturut-turut karena WHILE, FOR TO, dan FOR DOWNT0 semuanya menghitung penjumlahan 1 sampai 5 yang hasilnya sama, hal ini juga memverifikasi bahwa ketiga implementasi loop menghasilkan hasil yang konsisten. Nilai 120 merupakan hasil faktorial 5 yang dihitung melalui kombinasi fungsi dan FOR loop.

3.2.17. Kasus Uji 17

- Kasus: Program berhasil melalui uji kasus ini. Setiap baris output sesuai dengan hasil yang diharapkan. Nilai 15 muncul tiga kali berturut-turut karena WHILE, FOR TO, dan FOR DOWNT0 semuanya menghitung penjumlahan 1 sampai 5 yang hasilnya sama, hal ini juga memverifikasi bahwa ketiga implementasi loop menghasilkan hasil yang konsisten. Nilai 120 merupakan hasil faktorial 5 yang dihitung melalui kombinasi fungsi dan FOR loop.
- Tujuan: Kasus ini bertujuan untuk memastikan instruksi JPC dan JMP yang di-generate oleh genIf sudah benar pada kondisi batas, memastikan

backpatching berfungsi dengan tepat pada nested IF, serta memastikan tidak ada instruksi yang tereksekusi secara tidak sengaja ketika kondisi bernilai false dan tidak ada blok else.

- Input:

```
program TC17;
var
  x : integer;
begin
  { Edge 1: kondisi tepat di batas (boundary) }
  x := 5;
  if x > 5 then
    writeln(1)
  else
    writeln(0);

  { Edge 2: kondisi false, tidak ada else (tidak boleh ada output) }
  x := 3;
  if x > 10 then
    writeln(99);

  { Edge 3: nested if, semua cabang false kecuali else terdalam }
  x := 1;
  if x > 10 then
    writeln(3)
  else
    if x > 5 then
      writeln(2)
    else
      writeln(1);

  { Edge 4: nested if, cabang pertama true (tidak masuk else) }
  x := 15;
  if x > 10 then
    writeln(3)
  else
    if x > 5 then
      writeln(2)
    else
      writeln(1);

  { Edge 5: kondisi dengan ekspresi kompleks }
  x := 6;
  if x > 5 then
    if x < 10 then
```

```
writeln(1)
else
  writeln(0)
else
  writeln(0)
end.
```

- Output:

```
=== Output Program ===
0
1
3
1
```

- Analisis: Program berhasil melalui uji kasus ini. Edge 1 membuktikan bahwa operator perbandingan GTR (bukan GEQ) digunakan dengan benar sehingga nilai tepat di batas menghasilkan false. Edge 2 tidak menghasilkan output sama sekali, membuktikan bahwa JPC langsung melompat ke instruksi setelah blok then ketika tidak ada blok else. Edge 3 dan 4 membuktikan backpatching pada nested IF berjalan benar. Edge 5 membuktikan bahwa nested IF di dalam cabang then juga di-generate dengan benar.

3.2.18. Kasus Uji 18

- Kasus: Program menguji empat skenario perulangan WHILE: kondisi langsung false sehingga body tidak pernah dieksekusi, loop dengan tepat satu iterasi, loop dengan beberapa iterasi, dan loop dengan kondisi yang berubah secara eksponensial.
- Tujuan: Kasus ini bertujuan untuk memastikan evaluasi kondisi WHILE dilakukan sebelum body dieksekusi (pre-condition loop), memastikan JPC langsung melompati body ketika kondisi false sejak awal, dan memastikan JMP kembali ke awal loop dengan benar.
- Input:

```
program TC18;
var
  x : integer;
```

```

y : integer;
begin
  { Edge 1: kondisi langsung false, body tidak pernah dieksekusi }
  x := 10;
  y := 0;
  while x < 5 do
  begin
    y := y + 1;
    x := x + 1
  end;
  writeln(y);

  { Edge 2: loop hanya 1 iterasi }
  x := 4;
  y := 0;
  while x < 5 do
  begin
    y := y + 1;
    x := x + 1
  end;
  writeln(y);

  { Edge 3: nested while }
  x := 1;
  y := 0;
  while x <= 3 do
  begin
    y := y + x;
    x := x + 1
  end;
  writeln(y);

  { Edge 4: while dengan kondisi berubah di dalam body }
  x := 0;
  y := 1;
  while y < 100 do
  begin
    y := y * 2;
    x := x + 1
  end;
  writeln(x)
end.

```

- Output:

```

=== Output Program ===

```

```
0  
1  
6  
7
```

- Analisis: Program berhasil melalui uji kasus ini. Edge 1 menghasilkan 0 karena body tidak pernah dieksekusi, membuktikan bahwa WHILE adalah pre-condition loop. Edge 2 menghasilkan 1 karena hanya satu iterasi yang terjadi sebelum kondisi menjadi false. Edge 4 menghasilkan 7 karena nilai y digandakan sebanyak 7 kali (1, 2, 4, 8, 16, 32, 64, 128) sebelum melampaui 100, membuktikan bahwa kondisi WHILE dievaluasi ulang setiap iterasi dengan benar.

3.2.19. Kasus Uji 19

- Kasus: Program menguji lima skenario perulangan FOR: FROM sama dengan TO (tepat satu iterasi) untuk TO dan DOWNTO, iterasi banyak untuk TO dan DOWNTO, serta FOR dengan percabangan IF di dalam body.
- Tujuan: Kasus ini bertujuan untuk memastikan FOR TO dan FOR DOWNTO menghasilkan hasil yang simetris, memastikan loop berjalan tepat satu kali ketika nilai FROM sama dengan TO, dan memastikan statement kompleks di dalam body FOR di-generate dengan benar.
- Input:

```
program TC19;  
var  
  i : integer;  
  y : integer;  
begin  
  { Edge 1: for TO, from = to (tepat 1 iterasi) }  
  y := 0;  
  for i := 5 to 5 do  
    begin  
      y := y + i  
    end;  
  writeln(y);  
  
  { Edge 2: for DOWNTO, from = to (tepat 1 iterasi) }
```

```

y := 0;
for i := 3 downto 3 do
begin
  y := y + i
end;
writeln(y);

{ Edge 3: for TO, iterasi banyak }
y := 0;
for i := 1 to 10 do
begin
  y := y + i
end;
writeln(y);

{ Edge 4: for DOWNTO, iterasi banyak }
y := 0;
for i := 10 downto 1 do
begin
  y := y + i
end;
writeln(y);

{ Edge 5: for dengan body yang lebih kompleks (if di dalam for) }
y := 0;
for i := 1 to 5 do
begin
  if i > 2 then
    y := y + i
  end;
  writeln(y)
end.

```

- Output:

```

=== Output Program ===

```

```

5
3
55
55
12

```

- Analisis: Program berhasil melalui uji kasus ini. Edge 1 dan 2 membuktikan bahwa FOR dengan FROM sama dengan TO mengeksekusi body tepat satu kali. Edge 3 dan 4 menghasilkan nilai yang sama yaitu 55 (penjumlahan 1 sampai 10), membuktikan bahwa FOR TO dan FOR

DOWNTO menghasilkan hasil yang konsisten. Edge 5 menghasilkan 12 karena hanya nilai i yang lebih besar dari 2 (yaitu 3, 4, dan 5) yang dijumlahkan, membuktikan bahwa IF di dalam FOR di-generate dengan benar.

3.2.20. Kasus Uji 20

- Kasus: Program menguji tiga skenario perulangan REPEAT-UNTIL: kondisi terpenuhi langsung pada iterasi pertama sehingga loop hanya berjalan sekali, loop dengan banyak iterasi, dan loop dengan percabangan IF di dalam body.
- Tujuan: Kasus ini bertujuan untuk memastikan REPEAT selalu mengeksekusi body minimal satu kali berbeda dengan WHILE, memastikan JPC yang mengarah kembali ke awal loop berfungsi dengan benar, dan memastikan statement kompleks di dalam body REPEAT di-generate dengan benar.
- Input:

```
program TC20;
var
  x : integer;
  y : integer;
begin
  { Edge 1: kondisi langsung true di iterasi pertama (1 kali jalan) }
  x := 5;
  y := 0;
  repeat
    y := y + 1;
    x := x + 1
  until x == 6;
  writeln(y);

  { Edge 2: banyak iterasi sebelum kondisi terpenuhi }
  x := 1;
  y := 0;
  repeat
    y := y + x;
    x := x + 1
  until x == 10;
  writeln(y);
```

```

{ Edge 3: repeat dengan if di dalamnya }
x := 1;
y := 0;
repeat
  if x > 5 then
    y := y + 10
  else
    y := y + 1;
    x := x + 1
until x == 8;
writeln(y)
end.

```

- Output:

```

=== Output Program ===
1
45
25

```

- Analisis: Program berhasil melalui uji kasus ini. Edge 1 menghasilkan 1 karena body dieksekusi tepat satu kali sebelum kondisi terpenuhi, membuktikan sifat post-condition dari REPEAT. Edge 2 menghasilkan 45 karena menjumlahkan 1 sampai 9. Edge 3 menghasilkan 25 karena untuk x bernilai 1 sampai 5 nilai y bertambah 1 per iterasi (total 5), sedangkan untuk x bernilai 6 dan 7 nilai y bertambah 10 per iterasi (total 20), sehingga y akhir adalah 25. Loop berhenti ketika x mencapai 8 setelah iterasi ke-7.

3.2.21. Kasus Uji 21

- Kasus: Program menguji lima skenario pernyataan CASE-OF: nilai cocok di branch pertama, nilai cocok di branch terakhir, nilai tidak cocok dengan branch manapun, CASE dengan satu branch saja, dan CASE dengan body berupa compound statement.
- Tujuan: Kasus ini bertujuan untuk memastikan hanya satu branch yang dieksekusi ketika ditemukan kecocokan, memastikan seluruh branch dilewati ketika tidak ada yang cocok, dan memastikan JMP di akhir setiap body branch melompat ke instruksi setelah blok CASE dengan benar.

- Input:

```
program TC21;
var
  x : integer;
  y : integer;
begin
  { Edge 1: nilai cocok di branch pertama }
  x := 1;
  case x of
    1: writeln(10);
    2: writeln(20);
    3: writeln(30)
  end;

  { Edge 2: nilai cocok di branch terakhir }
  x := 3;
  case x of
    1: writeln(10);
    2: writeln(20);
    3: writeln(30)
  end;

  { Edge 3: nilai tidak cocok dengan branch manapun (tidak ada output) }
  x := 9;
  y := 0;
  case x of
    1: y := 1;
    2: y := 2;
    3: y := 3
  end;
  writeln(y);

  { Edge 4: case dengan satu branch saja }
  x := 1;
  case x of
    1: writeln(42)
  end;

  { Edge 5: case dengan body compound statement }
  x := 2;
  y := 0;
  case x of
    1: begin
        y := 10;
```

```
writeln(y)
end;
2: begin
  y := 20;
  writeln(y)
end
end
end.
```

- Output:

```
=== Output Program ===
10
30
0
42
20
```

- Analisis: Program berhasil melalui uji kasus ini. Edge 1 membuktikan branch pertama terdeteksi dengan benar. Edge 2 membuktikan seluruh branch dievaluasi secara berurutan hingga branch terakhir ditemukan. Edge 3 menghasilkan 0 karena tidak ada branch yang cocok sehingga variabel y tidak pernah diubah dari nilai awalnya, membuktikan bahwa seluruh branch dilewati dengan benar. Edge 4 membuktikan CASE dengan satu branch berfungsi. Edge 5 menghasilkan 20 karena compound statement di branch 2 dieksekusi secara lengkap.

3.2.22. Kasus Uji 22

- Kasus: Program menguji delapan skenario pemanggilan prosedur dan fungsi: prosedur tanpa parameter, prosedur dengan satu parameter, prosedur dengan tiga parameter, prosedur dengan ekspresi sebagai argumen, prosedur yang mengandung loop di dalamnya, function call di dalam ekspresi assignment, function call bersarang (fungsi memanggil fungsi lain), dan function call langsung di dalam writeln.
- Tujuan: Kasus ini bertujuan untuk memastikan mekanisme passing argumen ke dalam frame prosedur dan fungsi berjalan benar, memastikan return value fungsi dikembalikan ke caller dengan tepat, dan memastikan pemanggilan bersarang menghasilkan hasil yang benar.

- Input:

```
program TC22;
var
  x : integer;
  y : integer;

procedure tanpaParam;
begin
  writeln(99)
end;

procedure satuParam(a : integer);
begin
  writeln(a)
end;

procedure tigaParam(a : integer; b : integer; c : integer);
begin
  writeln(a);
  writeln(b);
  writeln(c)
end;

procedure denganLoop(n : integer);
var
  i : integer;
begin
  i := 1;
  while i <= n do
  begin
    write(i);
    i := i + 1
  end;
  writeln(0)
end;

function kali(a : integer; b : integer) : integer;
begin
  kali := a * b
end;

function kuadrat(n : integer) : integer;
begin
  kuadrat := kali(n, n)
end;
```

```

begin
  { Edge 1: prosedur tanpa parameter }
  tanpaParam;

  { Edge 2: prosedur dengan satu parameter }
  satuParam(42);

  { Edge 3: prosedur dengan tiga parameter }
  tigaParam(1, 2, 3);

  { Edge 4: prosedur dengan ekspresi sebagai argumen }
  x := 3;
  y := 4;
  satuParam(x + y);

  { Edge 5: prosedur dengan loop di dalamnya }
  denganLoop(5);

  { Edge 6: function call di dalam ekspresi }
  x := kali(3, 4);
  writeln(x);

  { Edge 7: function call bersarang }
  x := kuadrat(5);
  writeln(x);

  { Edge 8: function call langsung di writeln }
  writeln(kali(6, 7))
end.

```

- Output:

```

=== Output Program ===
99
42
1
2
3
7
123450
12
25
42

```

- Analisis: Program berhasil melalui uji kasus ini. Edge 1 membuktikan prosedur tanpa parameter dapat dipanggil dengan benar. Edge 3

membuktikan tiga argumen diteruskan ke dalam frame dengan urutan yang benar. Edge 4 menghasilkan 7 karena ekspresi $x+y$ dievaluasi di sisi caller sebelum diteruskan sebagai argumen, membuktikan bahwa `genExpression` dipanggil dengan benar untuk setiap argumen. Edge 5 menghasilkan 123450 karena `write` mencetak 1 hingga 5 tanpa newline kemudian `writeln(0)` menambahkan newline. Edge 7 menghasilkan 25 karena `kuadrat(5)` memanggil `kali(5, 5)` secara internal, membuktikan bahwa function call bersarang menghasilkan return value yang benar melalui mekanisme stack frame berlapis.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan pada Milestone 4, dapat ditarik beberapa kesimpulan sebagai berikut:

- Program ICG untuk bahasa pemrograman Arion telah berhasil diimplementasikan menggunakan bahasa pemrograman C++. Implementasi ini dibangun sesuai dengan spesifikasi yang diminta.
- Program telah diimplementasikan dengan Interpreter, Stack, VM, dan Penanganan Galat. Program telah mampu menjembatani kode sumber (AST terdekori) dengan instruksi mesin virtual (P-code/bytecode) dengan instruksi seperti LIT, LOD, STO, CAL, OPR, RET, LODA, STOA, CKB). Program telah mampu memanggil instruksi (prosedur/fungsi) dan mengalokasikannya dalam sebuah stack frame.

4.2. Saran

Untuk pengembangan selanjutnya, terutama untuk milestone berikutnya, terdapat beberapa saran yang dapat diterapkan. Program saat ini melakukan pengecekan batas untuk alamat yang berada di dalam stack. Jika suatu variabel mengakses alamat memori di luarnya (misal, dengan mengakses alamat miliknya-1 yang masih valid), program akan tetap berjalan. Program saat ini belum mampu menangani kasus ketika index bernilai negatif. Program dapat dieksploitasi dengan menggunakan tipe data tertentu dengan array berindeks negatif. Array ini nantinya dapat mengakses elemen lain di stack selama masih berada di dalamnya. Hal ini dapat menyebabkan stack corruption atau overwrite isi stack (seperti pada Kasus Uji 1). Untuk perbaikan kedepannya, ICG dapat memvalidasi apakah offset field target berada dalam rentang ukuran miliknya (agar tidak mengakses variabel lain yang masih valid di dalam stack). Selain itu, dapat ditambahkan mekanisme pengecekan seperti *canary* untuk mencegah *stack corruption* terutama untuk *return value*.

DAFTAR PUSTAKA

- Spivak, R. (2015, December 15). Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees. Ruslan's Blog. Retrieved from <https://ruslanspivak.com/lsbasi-part7>
- Crafting Interpreters. (n.d.). Parsing Expressions. Retrieved from <https://craftinginterpreters.com/parsing-expressions.html>
- Naufarrel. (2025, December 22). Pengenalan Context-Free Grammar (CFG) dan Derivasi. IF-Notes. Retrieved from [https://if-notes.naufarrel.dev/02.-Catatan/IF2224-Teori-Bahasa-Formal-dan-Otomata/UAS/Pengenalan-Context-Free-Grammar-\(CFG\)-dan-Derivasi](https://if-notes.naufarrel.dev/02.-Catatan/IF2224-Teori-Bahasa-Formal-dan-Otomata/UAS/Pengenalan-Context-Free-Grammar-(CFG)-dan-Derivasi)